

Master's Programme in Computer, Communication and Information Sciences

Hardening Application Deployment in Kubernetes

Dat Doan

© 2025

This work is licensed under a [Creative Commons](#)
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Dat Doan

Hardening Application Deployment in Kubernetes

Computer Science

Prof. Tuomas Aura

Msc. Jose Luis Martin Navarro

31 7 2025

69+6

English

Abstract

Containers and Kubernetes are widespread technologies that have evolved how applications are developed, deployed, and managed, offering portability, scalability, and resilience. However, the security challenges associated with Kubernetes deployments remain critical, especially given its popularity for hosting multi-tenant applications. Several frameworks try to guide the deployment of secure applications, but they are far from offering complete and fine-grained instructions on how to achieve a secure cluster. This creates uncertainty about what securing a deployment requires, leaving applications susceptible to a variety of attack vectors, such as misconfigured roles, exposed secrets, and unmonitored runtime behavior. Addressing these vulnerabilities demands a systematic application of established security practices tailored to Kubernetes environments.

The objective of this thesis is to deliver a robust, automated, and scalable security hardening framework for a Kubernetes cluster. This thesis proposes a new approach to implementing security practices systematically within a cloud and Kubernetes system. It covers most security measures and attack vectors while ensuring early attention to security through a two-dimensional matrix for container security practices. The implementation includes a wide range of configuration and best practices from standard industrial documents such as RBAC, network policy, configuration scanning, distroless image, and so on. The proposed approach is validated through a case study that aligns with different stages of the DevOps life cycle.

The results indicate two main points. First, a more comprehensive approach with a case study is presented to easily capture both the big picture and details about security in Kubernetes. Second, the study goes through the steps to see the feasibility of the new approach and shows its effectiveness in reducing 100% of high and critical severity vulnerabilities from the software and dependencies. This work contributes to improving the overall security posture of containerized applications. The methodology offers an improvement in the efficiency of security implementation for the Kubernetes cluster.

Kubernetes, Security, Containerization, Docker

Dat Doan

Sovellusten käyttöönoton vahvistaminen Kubernetesissa

Computer Science

Prof. Tuomas Aura

Msc. Jose Luis Martin Navarro

31 7 2025

69+6

Abstract

Kontit ja Kubernetes ovat laajalle levinneitä teknologioita, jotka ovat kehittäneet sovellusten kehittämistä, käyttöönottoa ja hallintaa tarjoten siirrettävyyttä, skaalautuvuutta ja vikasietoisuutta. Kubernetes-käyttöönottoon liittyvät tietoturvaasteet ovat kuitenkin edelleen kriittisiä, erityisesti ottaen huomioon sen suosion monivuokralaisten sovellusten ylläpidossa. Useat kehykset pyrkivät ohjaamaan turvallisten sovellusten käyttöönottoa, mutta ne ovat kaukana kattavista ja yksityiskohtaisista ohjeista turvallisen klusterin saavuttamiseksi. Tämä luo epävarmuutta siitä, mitä käyttöönoton suojaaminen vaatii, jolloin sovellukset ovat alttiita erilaisille hyökkäysvektoreille, kuten väärin määritetyille rooleille, paljastuneille salaisuuksille ja valvomattomalle ajonaikaiselle toiminnalle. Näiden haavoittuvuuksien korjaaminen edellyttää vakiintuneiden tietoturvakäytäntöjen systemaattista soveltamista, jotka on räätälöity Kubernetes-ympäristöihin. Tämän opinnäytetyön tavoitteena on tarjota vankka, automatisoitu ja skaalautuva tietoturvan vahvistamiskehys Kubernetes-klusterille. Tämä opinnäytetyö ehdottaa uutta lähestymistapaa tietoturvakäytäntöjen systemaattiseen toteuttamiseen pilvi- ja Kubernetes-järjestelmässä. Se kattaa useimmat turvatoimenpiteet ja hyökkäysvektorit varmistaen samalla varhaisen tietoturvan huomioimisen kaksiulotteisen matriisin avulla konttitietoturvakäytännöille. Toteutus sisältää laajan valikoiman konfiguraatioita ja parhaita käytäntöjä teollisuusstandardidokumenteista, kuten RBAC, verkkopolitiikka, konfiguraation skannaus, jakeluton levykuva ja niin edelleen. Ehdotettu lähestymistapa validoidaan tapaustutkimuksen avulla, joka on linjassa DevOps-elinkaaren eri vaiheiden kanssa. Tulokset osoittavat kaksi pääkohtaa. Ensinnäkin esitetään kattavampi lähestymistapa tapaustutkimuksen avulla, jotta voidaan helposti saada sekä kokonaiskuva että yksityiskohdat Kubernetesin tietoturvasta. Toiseksi tutkimus käy läpi vaiheet, jotta voidaan nähdä uuden lähestymistavan toteutettavuus, ja osoittaa sen tehokkuus ohjelmiston ja riippuvuuksien vakavien ja kriittisten haavoittuvuuksien 100% vähentämisessä. Tämä työ auttaa parantamaan konttisovellusten yleistä tietoturvatilannetta. Menetelmä tarjoaa parannuksen Kubernetes-klusterin tietoturvan toteutuksen tehokkuuteen.

Kubernetes, Turvallisuus, Containerization, Docker

Preface

Over the past six months, this thesis has been a journey of curious research and profound personal growth. After challenging hours of reading, writing, analysis, and fieldwork, I find the work presented in this report incredibly rewarding. Such an undertaking would not have been possible without the unwavering support of dedicated mentors. I wish to express my sincere gratitude to the individuals whose guidance and encouragement have been invaluable throughout this journey.

To my supervisor, Prof. Tuomas Aura, thank you for granting me the opportunity to work on this interesting topic and for providing invaluable ideas for my methodology. I extend my particular appreciation to my advisor, Jose Luis Martin Navarro, who closely supported me throughout the process and offered insightful suggestions. Thank you for your time, expertise, and constructive criticism, both during and outside of academic hours.

Otaniemi, 31 July 2025

Dat Doan

Contents

Abstract	3
Abstract	4
Preface	5
Contents	6
Symbols and abbreviations	9
1 Introduction	11
2 Background	13
2.1 Containers and Docker	13
2.2 Kubernetes	14
2.2.1 Kubernetes Architecture	15
2.2.2 Pod Architecture	16
2.2.3 Service Discovery	16
2.2.4 Configuration Data	17
2.2.5 Customer Resource and Operators	17
2.3 DevOps	18
2.4 Security Approaches	18
2.4.1 Threat Modeling	19
2.4.2 Security Principles	19
2.4.3 Security Frameworks	21
2.4.4 Threat Matrix for Kubernetes	21
2.4.5 Kubernetes CVEs	23
3 Literature Review	24
3.1 Cluster Security	24
3.1.1 Namespace Boundaries	24
3.1.2 Secrets Management	24
3.1.3 Authentication and Authorization	25
3.1.4 Role-based Access Control	26
3.1.5 Network Policies	26
3.1.6 Namespace Resource Quotas	27
3.1.7 Dynamic Admission Control	28
3.2 Workload and Pod Level Security	28
3.2.1 Pod Security Standards and Pod Security Admission	28
3.2.2 Deploy Honeypot Workload and Intrusion Detection Systems	29
3.2.3 Static Analysis of YAML file and configuration	30
3.2.4 Resource Management for Pods	31
3.3 Container Security	31
3.3.1 Build Secure Image	31

3.3.2	Image Scanning	32
3.3.3	Container Threat Analysis	33
3.3.4	LimitRanger	34
3.4	Monitoring, Alerting and Auditing	34
3.5	Compliance Check	36
3.5.1	CIS Security Benchmark	36
3.5.2	Kubernetes Hardening Guide	36
3.5.3	Related Work in Compliance Check	37
3.6	Research Gaps	39
4	Methodology	40
4.1	Methodology Overview	40
4.2	Matrix Design	41
4.2.1	Layer Definition	41
4.2.2	Stage Definition	42
4.2.3	Item Definition	43
4.3	How to Apply When Building And Releasing Software	43
4.3.1	Layer By Layer	43
4.3.2	Stage By Stage	44
4.3.3	Agile Implementation	44
5	Case Study And Result	45
5.1	Application Overview	45
5.1.1	Dealer	46
5.1.2	Guard	46
5.1.3	User	47
5.1.4	Access	47
5.1.5	Wiki	48
5.1.6	Environment Setup	48
5.2	Container Layer	49
5.3	Resources Layers	51
5.4	Workload Layers	53
5.5	Cluster Layer	55
6	Discussion	59
6.1	Case Study Output Result	59
6.2	Challenges	59
6.3	Limitation and Future Work	60
7	Conclusion	62
	References	63
A	Resource Management for Pods	70
B	LimitRanger	70

Symbols and abbreviations

Abbreviations

ABAC	Attribute-based access control
AKS	Azure Kubernetes Service
API	Application Programming Interface
ARA2	Anonymous Registration Anonymous Access
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
AVD	Aqua Vulnerability Database
CI/CD	Continuous Integration/Continuous Delivery
CISA	Cybersecurity and Infrastructure Security Agency
CIS	Center for Internet Security
CLIP	Contrastive Language-Image Pre-training
CNCF	Cloud Native Computing Foundation
CNI	Container network interface
CPU	Central Processing Unit
CRD	Custom Resource Definition
CVEs	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DevOps	Development and Operation
DevSecOps	Development, Security and Operations
DOS	Denial of Service
EKS	Amazon Elastic Kubernetes Service
GKE	Google Kubernetes Engine
GPT	Generative Pre-trained Transformer
GPU	Graphics processing unit
HIPAA	Health Insurance Portability and Accountability Act
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
IKS	IBM Cloud Kubernetes Service
IP	Internet Protocol
I/O	Input/Output
K8S	Kubernetes
MB	Megabyte
MITRE	Massachusetts Institute of Technology Research and Engineering
MVP	Minimum Viable Product
NIST	National Institute of Standards and Technology
NSA	National Security Agency
NVD	National Vulnerability Database
OS	Operating System
OAuth	Open Authorization
OWASP	Open Worldwide Application Security Project

PSA	Pod Security Admission
PSS	Pod Security Standards
RAM	Random Access Memory
RBAC	Role-Based Access Control
SSH	Secure Shell
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
URL	Uniform Resource Locator
VM	Virtual Machine

1 Introduction

In recent years, cloud-based applications have been developed with the support of various frameworks and technologies. Kubernetes and Docker are the mainstream in virtualization technology with 84% usage in 2023 [1]. Docker is an open-source technology that encourages developers to build, deploy, run, update, and handle containers, and Kubernetes is a portable, extensible platform for operating containerized applications [2, 3]. These technologies simplify the deployment, increase the reusability and scalability of microservices systems. Given their widespread adoption, the related security concerns increase, for example [4]. Therefore, organizations such as OWASP and NSA research and provide their best practices to secure cloud deployments [5, 6].

The complex nature of cloud environments, which involve multiple abstraction layers and technologies, makes it harder to completely secure both applications and infrastructure. While there are tools and hardening guides available to protect Kubernetes clusters, there is a lack of a comprehensive framework, leaving practitioners the task of conveying all the fragmented knowledge into a list of actionable steps needed to harden the cluster and applications.

The main objective of this thesis is to create a hardening framework based on previous state-of-the-art guidelines. This research combines the best existing methods to produce a strategy to reduce the threat exposure and limit the number of vulnerabilities. It groups common practices into a conceptual matrix that can be used to guide the developer's efforts.

To illustrate the practicality of the research, the framework is applied to a case study on a Kubernetes cluster. The case study includes the hardening of the cluster but also the security steps involving the application and its containers. The end result is analyzed with a focus on the number of vulnerabilities mitigated.

This thesis focuses on building the most comprehensive and complete security process for a typical Kubernetes cluster. The guideline covers most of the attack vectors in the cluster with fine-grained detail that teams can easily follow and apply to production environments. The research incorporates existing state-of-the-art approaches, tools, and configuration practices. Given the large number of guides and individual steps present in the literature review, this thesis leverages those with a major impact to form a new framework.

The scope of this thesis is limited to Kubernetes and container technologies, focusing on Docker. The research focuses on securing a general Kubernetes cluster and is cloud provider agnostic. A more complete solution would require an extension to cover specific cloud provider solutions and configurations. The developed method takes into account the real-world software development process and makes it easy for teams in charge of one step of the life cycle to understand what security steps are needed.

This thesis is organized as follows. Chapter 2 presents the background around the related topic. Chapter 3 reviews the current state of the art practices being used in Kubernetes security. Chapter 4 explains the proposed method and clarifies broken-down items. Chapter 5 examines the case study, evaluating the method using qualitative and quantitative results. Chapter 6 discusses the feasibility and adaptability of the new

approach in software development, challenges applying the scientific process to the case study, and the future work for improvements. Finally, chapter 7 summarizes the work and draws a conclusion from the process.

2 Background

This chapter covers basic concepts and technologies related to containers and their orchestration. Section 2.1 covers the basic information about containers and Docker. Section 2.2 describes Kubernetes, the most used technology to orchestrate containers, and the focus of this work. Section 2.3 defines DevOps, a modern software development approach, which is the idea for the later proposed method. Finally, section 2.4 introduces threat modeling, a structured approach to analyzing potential threats in Kubernetes, and several security principles to create a hardening strategy when developing a Kubernetes system.

2.1 Containers and Docker

Virtualization is the abstraction of computer resources from physical hardware [7]. Before the popularity of containers and Docker, virtualization technologies were already common. For example, virtual machines, the ancestor of containers, are an approach that provides an efficient, isolated replica of a computer system's environment [8]. Each virtual machine has its own OS and system libraries. Popular virtual machine providers are Microsoft Hyper-V, VirtualBox, and VMware [9, 10, 11]. The approach of virtual machines for virtualization provides diverse and well-established software environments. However, a major issue with the traditional method is the performance of resource-intensive, high storage use, and slow start-up time.

In contrast, using containers is the approach of visualizing only the OS, not the whole computer. Even though this technology has been invented, it became popular and an industrial standard only with Docker, a platform that supports developers to create and manage containers effectively and deliver the software faster. The container is a standard software unit that packages code along with all its dependencies, ensuring the application runs quickly and reliably across different computing environments [12]. The containers share the OS kernel, binaries, and libraries. There are many improvements compared to VM, such as a lightweight process, native performance, and a short startup time. MinSu et al. demonstrate that Docker container outperforms virtual machines in resource utilization, memory usage and disk I/O [7]. The packaged code and dependencies can work in any environment, including local, development, staging, and production. In Figure 1, Docker's architecture consists of

- Client: The Docker client allows users to interact with the Docker daemon through its API.
- Docker host: Docker host manages objects such as images and containers using the Docker daemon. Docker daemon listens to API requests from the client and interacts with the registry to get images. Docker daemon does most of the heavy tasks, such as pulling images from the registry, building, and running containers.
- Registry: This is remote storage for images. To start containers, the Docker daemon pulls images from the registry in case images are not available in local

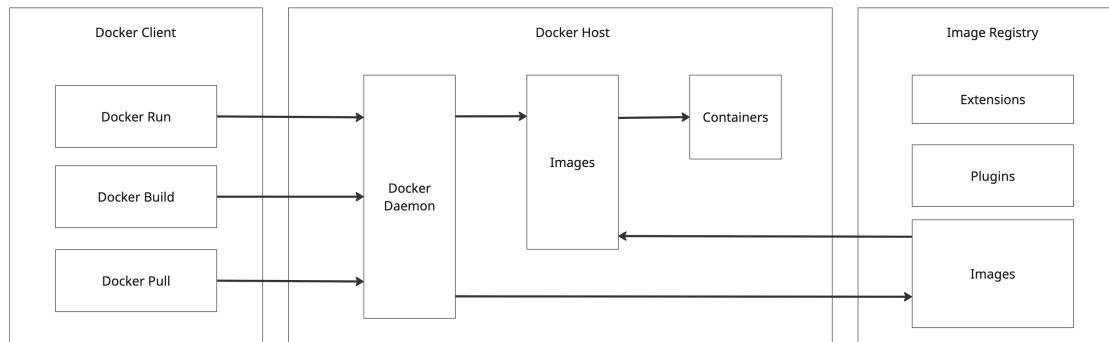


Figure 1: Docker architecture [13]

storage.

2.2 Kubernetes

The classic approach to managing running services involves using bare metal machines. Over time, containers have improved this process, allowing developers to easily transfer containers from their local machines to a server for deployment. Docker Compose is a tool that helps manage these containers, but it often requires many manual adjustments from the administrator [14]. Kubernetes, a container orchestration tool, was later developed to solve this problem. This orchestration system supports automating deployment, horizontal scaling, role-based access control (RBAC), and managing the life cycle of the containers. Kubernetes is the most widely adopted orchestration platform. It was originally developed and introduced under the name Borg by Google and has been released as open source, and is currently maintained by the community of contributors [15].

The smallest logical unit of a Kubernetes cluster is *Pod*, a group of containers with shared storage and network resources. All containers within the same Pod share the same life cycle. The life cycle of a Pod includes a pending phase, a running phase, and finally, a succeeded or failed phase if the container terminates or fails. In the pending phase, the Pod is accepted by the cluster. It downloads images from repositories and waits for all containers to start. Then, in the running phase, the Pod is assigned to a *worker node*, and all required containers are running. Depending on the restart policy statically defined, a Pod would be restarted back to the running stage or stopped if there are issues detected by the container probes function. If a Pod runs a job, it will later have *succeeded* or *failed* status. Succeeded status is for jobs that finish and will not start again, while failed status happens when all containers stop with an error occurring [3].

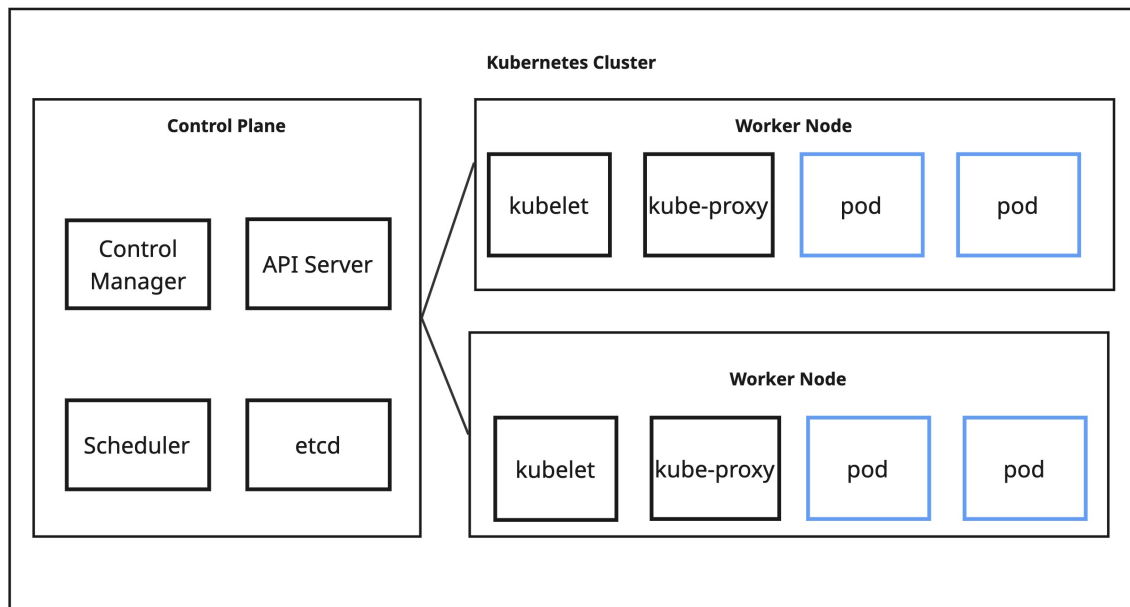


Figure 2: Kubernetes architecture

2.2.1 Kubernetes Architecture

A Kubernetes cluster comprises a *control plane* and multiple *worker nodes*. The control plane interacts with and manages all the worker nodes. In the control plane, there are different components: *control manager*, *Kubernetes API server* aka *kube-apiserver*, *scheduler*, key-value database *etcd*. First, *kube-apiserver*'s main function is to expose the Kubernetes API for the cluster. It can be scaled horizontally using many instances to balance the workload [3]. It handles and validates all requests from worker nodes, developers, and cluster administrators [16]. Second, *etcd* is a consistent and highly available key-value store for the whole cluster [3]. It persistently stores internal state, metadata, secrets, and configuration data [16]. *kube-apiserver* is the only component that communicates and updates states in *etcd*. In addition, the *kube-scheduler* has the role of allocating a node to a Pod.

The worker node is where the applications defined in each Pod run. In addition, two other Kubernetes components in the worker node are the *kubelet* and *kube-proxy*. Kubelet manages Pods, communicates with the control plane, manages the Pod life cycle, monitors the node's health, and manages the volumes. Additionally, *kube-proxy* manages services, balances message loading, and handles service discovery

Kubernetes uses a declarative approach to manage all resources inside the cluster. The developer teams do not need to care about creating, deploying, or managing *workload* and resources. Everything works under the abstraction and configuration of Kubernetes resources. To understand how Kubernetes operates applications and the whole system, the next subsections discuss fundamental resources used in Kubernetes and related security challenges.

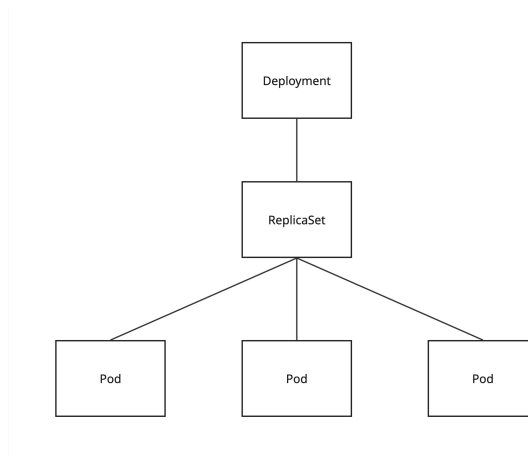


Figure 3: Example of Deployment architecture, with a workload replicated into three Pods. [3]

2.2.2 Pod Architecture

In a Kubernetes system, a Pod is a basic deployable unit. All applications run inside the Pod. However, naked Pod usage is not recommended because it can't be rescheduled in case of node failure. A workload object is created to abstract and manage a Pod, and it is easier to manage a Pod using a workload. Workload consists of *Deployment*, *StatefulSet*, *DaemonSet*, *Job* and *CronJob*. Deployment is the most common choice among the workloads to run stateless applications. A Deployment's architecture is described [Figure 3](#) with a lower layer of ReplicaSet that keeps a desired number of Pods. For more advanced requirements using persistent data, StatefulSets support the management of stateful applications. Jobs and CronJobs are used to handle short-lived tasks.

2.2.3 Service Discovery

A single cluster can have a large number of Pods running. Although those Pods can be deployed in different machines (nodes), Kubernetes's goal is to hide this to the Pods, which don't need to understand the underlying processes. One of the challenges of service discovery is the size. Big companies' clusters may have thousands of nodes running in a cluster. For example, OpenAI runs 7500 nodes for a system of different AI models such as GPT, CLIP, DALL-E [17]. Another challenge is the flexibility of the workloads. While each Pod has its own IP address, it is not practical to use it for routing data. First, Pods are ephemeral since a single Pod can be failed, destroyed, and recreated anytime. Thus, the IP address of Pods could be different from the previous Pod version. In addition, when horizontally scaled, new Pods are created with new IP addresses, and there is no way to keep track of these Pods. Second, using a Pod's address does not support load balancing, and requests will not be distributed evenly to all healthy Pods.

Kubernetes has a mechanism for network abstraction called *Services*. The Services,

as opposed to Pods, have stable IP addresses and a load balancing function to Pod's loose coupling with Pod resources. Requests sent to an application would be routed to the service resource and then distributed to the Pod with the container running the application. Pods targeted by a Service are defined using *selectors* and *labels*. While selectors are defined in Services, labels are attached to Pods. They are both key-value pairs used by Services to identify the correct Pods. As a result, the requests are routed to the Services as a fixed endpoint, which hides the dynamic nature of Pods and their constant changes.

2.2.4 Configuration Data

In Kubernetes, *ConfigMap* and *Secret* are two objects used for data configuration. They are basically key-value data that has a decoupled life cycle from the Deployment. The main difference is that ConfigMaps store plain text data such as environment variables, configuration files or connection URLs, while Secrets store data in base64-encoded form. Data in Secret is confidential, such as passwords, API keys, SSH keys, and OAuth.

Secret data is not encrypted by default in the etcd database. Anyone with permission to access the API can fetch and modify the secret content. All data is stored in plain text, but Kubernetes offers some security options to enhance protection for secrets. To make data in a Secret object secure, the required steps from the main Kubernetes document are:

- Enable encryption at rest [3]
- Configure RBAC rules with least-privilege access [3]
- Limit access to Secrets to specific containers only [3]

2.2.5 Customer Resource and Operators

In addition to the main components such as workloads, services, and data, Kubernetes provides extensions for software running in the cluster. *Custom resource* is a powerful extension besides the default Kubernetes resource [3]. Customer resources are created using *Custom Resource Definition* (CRD) or *Aggregated API*. CRD allows defining new types of Kubernetes resources without requiring a new API server, while Aggregated APIs enable serving resources from multiple, independent API servers. Using both the Custom Resource Definition and the bound controller is the most familiar method to use an Operator. This provides the cluster ecosystem with more abilities, such as add-ons, community extensions [3].

Operators or the operator pattern are software extensions that utilize the customer resources for extended behaviors [3]. It creates a higher-level abstraction with functionality. Operators support automation in many tasks including deploying, restoring application state, or choosing a leader in a distributed environment. This is the complicated pattern with hard usage in the Kubernetes ecosystem, but really powerful. Common tools developed in the Kubernetes ecosystem are operator-based, such as Strimzi Kafka and Trivy Operator [18, 19].

2.3 DevOps

DevOps is the integration of cultural philosophies, practices, and tools that enhance an organization's power to deliver products and services quickly. This approach enables organizations to grow and improve their products at an impressive rate compared to those using traditional software development and infrastructure management processes. [20]. Development and operations teams collaborate throughout the entire application life cycle, from development and testing to deployment and operations. They develop a wide range of skills beyond a single function [20]. The common issue of a microservices system that leverages a Kubernetes scaling environment is the requirement of rapid, safe, and reliable delivery of these complex microservices. DevOps practices such as continuous integration and continuous delivery solve this issue. It is widely adopted now and is a part of the modern software development process with microservices and Kubernetes as standard practices and technology choices. Different from the traditional and sequential Waterfall process, the DevOps life cycle is a system of parallel and continuous loop of constant flow. Constant feedbacks from operations are immediately instructed for the next development cycle. The DevOps life cycle is typically broken down into several stages [21]:

- **Plan:** Define the requirements, create a road map, and plan the development process. This stage involves gathering feedback, prioritizing tasks, and setting goals.
- **Code:** Developers write code to implement features or fix bugs, adhering to coding standards and best practices.
- **Build:** Compile the code, package it into artifacts, and prepare it for testing or deployment.
- **Test:** Validate the code and artifacts to ensure they meet quality standards and function as expected.
- **Release:** Package the tested code into a deployable artifact and prepare it for deployment to production or staging environments.
- **Deploy:** Deploy the application or updates to production or staging environments, often using automated processes.
- **Operate:** Manage and maintain the application in production, ensuring it runs smoothly and efficiently
- **Monitor:** Continuously observe the application and infrastructure to ensure performance, availability, and security.

2.4 Security Approaches

By default, the dynamic environment of Kubernetes exposes many security risks, and securing this is challenging. Security frameworks and guidelines are commonly

used in industry to cover vulnerabilities for the best security practices. It supports understanding the best practices for attack and defense.

2.4.1 Threat Modeling

In the context of Kubernetes security, threat modeling is an effective way to determine possible threats and examine the effect of those threats. Furthermore, this helps to mitigate and prevent common threats. NSA describes three primary categories of threats in Kubernetes clusters and their sources of compromise, including *supply chain*, *malicious threat actor*, and *insider threat* [5]. In the scope of this thesis, the primary threats are the supply chain and the malicious threat actor.

- **Supply chain:** Supply chain cyberattack is carried out to gain unauthorized access to the cluster. This is conducted by varied attack vectors and is challenging to defend against. This can impact multiple levels of the Kubernetes cluster, including:
 - Container/application level: Not all containers are secure. Attackers can perform initial access tactics that exploit malicious containers and containers with malicious dependencies.
 - Container runtime: This is the software that manages system resources, separates and executes containers on the node. The vulnerabilities in containers lead to inadequate isolation between containers.
 - Infrastructure: The cluster is not secured if there is a foothold in the underlying system hosting that cluster. The vulnerabilities may come from the software and hardware dependencies.
- **Malicious threat actor:** Malicious threat actor in the context related to Kubernetes is a person or organization that causes harm to the cluster and exploits the inside components. The actors can target cluster-exposed APIs, including:
 - Control plane: Control plane components always expose APIs for communication. There are many attack surfaces that can be exploited from the unprotected APIs.
 - Worker nodes: Worker nodes also have many components used for communication, such as kubelet and kubeproxy. Cyber actors can take advantage of these components to access the running nodes.
 - Containerized applications: Applications deployed and running in the cluster can be accessed from outside the cluster. An attacker can later escalate privileges in the Kubernetes environment.

2.4.2 Security Principles

Security principles are the highest-level abstraction concepts and philosophies used for decision-making in hardening systems. According to Liz et al., three principles

can be used to implement a security framework for Kubernetes, including defense in depth, the principle of least privilege, and limiting the attack surface [22]. They work together to reduce risk and ensure strong protection. Defense in depth protects layers, and the principle of least privilege limits the access for components in each layer, and finally, the limiting the attack surface principle narrows down the public entry points.

Defense in Depth

Defense in Depth is a security system that integrates individuals, technology, and capabilities to establish various protections across separate layers of the system [23]. A Kubernetes cluster has many implementation layers built on top of each other. A threat actor has varied choices for attacking a cluster. They can attack the containerized application or the workload running in that container at a higher level. In addition, the control plane and worker nodes are also ideal targets for exploitation. A single vulnerability can be exploited effectively without a solid defense. Due to this, there must be multiple layers of defense for a cluster.

Principle of Least Privilege

To limit the damage caused by security, the principle of least privilege demands the most restrictive set of privileges possible to perform a task [24]. Kubernetes components should be granted with the least permissions to function and operate [25]. Therefore, an actor with access to over-granted permissions resources can perform unauthorized actions using these permissions. Combined with the defense in depth principle, there must be several configurations to limit permissions and reduce privileges in different layers.

Limiting the Attack Surface

Pratyusa et al. defined attack surface as the set of ways in which an adversary can enter the system and potentially cause damage [26]. In Kubernetes, components expose APIs for communication, especially the kube API and the kubelet. Even though these components are not revealed to the external side, limiting the attack surface on them restrains the chance of lateral movement. Moreover, endpoints for sending requests to Pods have also been published. On top of that, more and more access points would be opened as the cluster size grows. Consequently, this reveals more attack vectors into a cluster.

Shift Left Security

Shift left security was a philosophy proposed by Larry Smith in 2021. This concept focuses on security processes that are conducted in the early stages of development rather than waiting until the end [27]. Shift left security can be applied in the context of Kubernetes and DevOps using best practices in each stage listed in 2.3. Conducting the security processes in the early stage may require effort, but it is necessary because the Kubernetes system is more complex than the traditional one. More specifically,

this emphasizes the effectiveness of doing security as early in the life cycle as possible. For example, it is easier to detect a vulnerability by statically analyzing a container before its deployment than trying to detect something erroneous at runtime. This approach allows for catching the security problems in the early stages and reduces the management of these issues in later stages [27].

2.4.3 Security Frameworks

Currently, there are several available frameworks for Kubernetes security. The frameworks include a set of guidelines and best practices to manage risk systematically. Common standard frameworks are the 4C's model, NIST SP 800-190, and the Kubernetes Hardening Guide from the National Security Agency [3, 28, 5].

The 4C's model stand for *Cloud*, *Clusters*, *Containers*, and *Code*. This framework aims to build a security process for these layers. For the Cloud layer, it focuses on infrastructure in Kubernetes, such as the control plane, nodes, etcd, and the API. Moreover, the security of the cloud provider is also a concern. The Cluster layer works on cluster components and the applications running. This layer has a list of areas including authentication, RBAC, network policies, secrets management, and Pod security standards. The Container layer is generally about container and OS vulnerabilities, container runtime isolation, image signing and enforcement, and disallowing unprivileged users. The final layer is the Code layer. However, this is out of scope for Kubernetes [3].

The NIST SP 800-190 document primarily works for application container technologies. It raises a list of major risks for the core components and suggests countermeasures for the risks. Unfortunately, this framework has a limited scope that only works for the containers running inside the Kubernetes cluster [28].

The Kubernetes Hardening Guide document is a security document for administrators and developers of United States national security systems. It covers security issues associated with improper configuration [5]. All the frameworks and guides above are valuable for use in real cloud systems. They help to reduce challenges for the complex nature of Kubernetes and document the best practices at the moment. The Kubernetes team also provides a list of security practices that focus on control plane protection, secrets, workload protection, admission control, and auditing [3].

2.4.4 Threat Matrix for Kubernetes

MITRE ATT&CK is a global knowledge base of adversary strategies derived from application observations. The ATT&CK is employed as a basis for designing threat models and methodologies for diverse organizations. [29]. There are currently 211 techniques with 468 sub-techniques in the Enterprise ATT&CK matrix at the time this thesis is conducted, and MITRE continuously adds more. The techniques describe how adversaries perform attack actions.

The ATT&CK matrix is the foundation for the Kubernetes threat matrix, which can be seen in Figure 4. The Kubernetes threat matrix is a simplified version that was created by the Azure Security Center. This matrix consists of attack techniques that

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Impact
Using Cloud credentials	Exec into container	Backdoor container	Privileged container	Clear container logs	List K8S secrets	Access the K8S API server	Access cloud resources	Data Destruction
Compromised images in registry	bash/cmd inside container	Writable hostPath mount	Cluster-admin binding	Delete K8S events	Mount service principal	Access Kubelet API	Container service account	Resource Hijacking
Kubeconfig file	New container	Kubernetes CronJob	hostPath mount	Pod / container name similarity	Access container service account	Network mapping	Cluster internal networking	Denial of service
Application vulnerability	Application exploit (RCE)		Access cloud resources	Connect from Proxy server	Applications credentials in configuration files	Access Kubernetes dashboard	Applications credentials in configuration files	
Exposed Dashboard	SSH server running inside container					Instance Metadata API	Writable volume mounts on the host	
							Access Kubernetes dashboard	
							Access tiller endpoint	

Figure 4: Threat matrix for Kubernetes [30]

are relevant to orchestration security in Kubernetes [30]. This matrix is usable for identifying attack vectors and prioritizing defenses for any Kubernetes system.

The main techniques include:

- **Initial Access:** Initial access techniques are attack methods to gain access to the resources. Attackers can gain this access through cloud credentials, compromised images, vulnerable applications, and exposed interfaces.
- **Execution:** Execution techniques are methods used to run malicious code inside a cluster. Attackers can use malicious commands, run their own container, and run code inside the deployed application.
- **Persistence:** Persistence techniques are used to keep the attackers with access to the cluster. Attackers may do it using backdoor containers, hostPath volume mounts, a directory, and Kubernetes CronJobs to run malicious code.
- **Privilege escalation:** These techniques are used by attackers to get higher privileges in the environment than those they currently have [30]. This can be done by accessing a privileged container or by having high permissions to resources such as hostPath mounts and cluster-admin bindings.
- **Defense evasion:** Defense evasion is a technique used to hide the activities of attackers. Evidence such as container logs and Kubernetes events is cleared. Attackers can also create similarly named backdoor Pods that look identical to Kubernetes' main components.
- **Credential access:** Attackers use credential access techniques to steal credentials, including Kubernetes Secrets, service accounts, and credentials in configuration files.

- **Discovery:** Attackers use discovery techniques to explore the environment to which they gained access.
- **Lateral movement:** Attackers use lateral movement to move through the victim's environment.
- **Impact:** Impact techniques lead to consequences such as data destruction, resource hijacking, and denial of service. This can be performed by destroying resources, configuration, and storage, and manipulating compromised resources.

2.4.5 Kubernetes CVEs

The *Common Vulnerabilities and Exposures* (CVEs) is a list of documented vulnerabilities and exposures commonly found in typical applications [16]. A CVE's record describes a vulnerability using a description, root cause, severity of the issue, and references to the details of the issue published by the researchers. Common Vulnerability Scoring System (CVSS) is used as scores assigned to a single CVE record [16]. The CVE public database could be effectively used to address security risks and reduce the attack surface of a Kubernetes cluster.

3 Literature Review

This chapter provides a literature review on Kubernetes cluster security best practices. The literature review is structured to analyze the best practices in diverse layers and components. Section 3.1 explores approaches for protecting a typical Kubernetes cluster. Section 3.2 delves deeper into safeguarding the workload and pod, which represent the core of the application. In the lower layer of workload, section 3.3 covers techniques to make Docker images and containerized applications safe for usage. Section 3.4 displays how to monitor and what to monitor for each component and each layer. Section 3.5 focuses on compliance checks and methods to protect components that operate the system, such as control plane components and worker node components. Finally, section 3.6 represents the current gap of these practices and what is missing for a team to apply in software development.

3.1 Cluster Security

At the highest cluster level, certain practices can influence the entire cluster. This section outlines various practices that all contribute to creating a secure cluster. These practices include secret management, authentication, authorization, and managing namespace resource quotas. Typically, these tasks fall under the responsibilities of the cluster administrator.

3.1.1 Namespace Boundaries

Kubernetes namespaces are meant to separate resources for multiple teams within a cluster. There are 3 default namespace including *kube-system*, *kube-public*, *default*. While *kube-system* and *kube-public* namespaces cannot be deleted, they should not be utilized to organize resources. Custom namespaces provide an initial layer of isolation between resources, especially useful in multi-tenant environments. Custom namespaces can facilitate resource isolation, effectively establishing a virtual segment with specific rules, resource usage, and network traffic controls. Additionally, developers can set up RBAC, resource limits, and more advanced configurations to enhance security [5]. However, Kubernetes Operators need privileges and may connect to resources across different namespaces [31]. Andong et al. identified vulnerabilities in cross-namespace references within Kubernetes Operator due to dissimilarity between the declared scope of a resource and the actual scope of its process logic [31]. The same research shows that 317 out of 2268 Operators on GitHub are vulnerable to cross-namespace attacks. Consequently, lateral movement is not prevented by default between namespaces without advanced configuration.

3.1.2 Secrets Management

Code running in the Kubernetes cluster generally retrieves secret data. These secret data, which are stored using secret objects by Kubernetes, can be API keys, OAuth tokens, and SSH keys [5]. Automatically, the resource is an etcd key-value database,

and it is not encrypted without configuration [3]. This is a common target in the attack vector. This subsection discusses methods to work with secret and secure sensitive information.

Secret Encryption And Secret Storage

Encryption is the primary choice to protect secret values. Arsh et al. study explains that using cryptography and encryption is a main way to secure data on the cloud [32]. In Kubernetes, secrets are insecurely stored in the etcd database in base64 encoding by default. It can be encrypted at rest and in transit [3]. Besides the etcd database, secret values can be stored in third-party solutions such as HashiCorp Vault. This third-party tool supports secret storage with strong encryption for data at rest, in use, and in transit, and offers many advanced features such as auto-rotating secrets and dynamic secrets.

Secret Rotation And Revocation

In secret management, secret rotation is the process of regularly updating the secret. Secret rotation is an important practice to protect sensitive data. A secret can be safe from human guessing, but not safe from machine attacks. Shortening the period of a secret makes brute force attacks harder and more complex. Even though the secrets are compromised, threat actors only have a limited time to perform unauthorized actions using the leaked key. This is not a built-in function of the Kubernetes system, but alternative third-party solutions, such as HashiCorp Vault, offer this function [22].

Secret revocation is equally vital. Secret revocation is the action of invalidating the secret. This helps to ensure that the old key or compromised secret can't be exploited. For example, after leaving the organization, an employee can't use the existing secret anymore.

Apart from the main methods for secrets protection using key and encryption, Shazibul et al. also recommend spreading sensitive workload to prevent escalation on a compromised node [33]. Kavindu et al. simplified secrets management with *K8s Pro Sentinel*, a GUI-based Operator that supports creating, modifying, and auditing secrets [34]. However, the related academic literature is still far from a comprehensive solution for secrets management.

3.1.3 Authentication and Authorization

Both *authentication* and *authorization* are very important for an application. While authentication is used to verify the identity of the actor, authorization is later used to verify the permission associated with the request [16]. In Kubernetes, the core component that exposes the API and verifies all requests is the kube-apiserver [16].

Authentication

Authentication is the initial step in Kubernetes security, verifying actors, services or applications that are trying to access the API. There are many mechanisms for

authentication, including static tokens, basic authentication with a password, service account tokens, bootstrap tokens, webhook tokens, authentication proxy tokens, client certificates, and user impersonation [16]. All methods can be enabled simultaneously.

Authorization

After authentication is the authorization process. The authorization module determines the permission of the identified user or service and returns the decision to allow or deny. There are many authorization modes, such as Node authorization, ABAC mode, RBAC, AlwaysDeny, AlwaysAllow, and webhooks [22]. Each option has its own pros and cons, but RBAC is the most widely used and recommended despite its complexity [25, 35]. AlwaysDeny mode blocks all the requests, and AlwaysAllow mode allows all requests. They should be used only for testing and development, not for the released production.

3.1.4 Role-based Access Control

Role-based access control(RBAC) is an authorization method that works based on the roles of users [25]. Roles with a set of permissions are created and assigned to users. The scopes of using RBAC are cluster-wide and namespace-wide [22]. The four core objects used for RBAC are Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings. There are use cases to use these objects:

- Use Roles and RoleBindings to create roles and assign them to users in the namespace scope.
- Reuse role with ClusterRoles in different namespaces and assign to users using RoleBindings.
- Use ClusterRoles to create cluster-wide permissions and assign to users using ClusterRoleBindings [22].

3.1.5 Network Policies

In the Kubernetes environment, the network model is flat, and all traffic is allowed in the cluster network. This means that every pod can communicate with any other pod in the cluster freely, even in different namespaces. This approach simplifies the network design in the cluster [25]. Network policies have been introduced to control the traffic between pods. Network policies are resources that define how traffic is routed in the network. They control what pods can send or receive requests.

Ingress and egress rules are the rules used to control the traffic. Ingress defines rules for incoming traffic, and egress defines rules for outgoing traffic. Consequently, the pod ingress traffic of the destination must match the egress traffic of the source. The best practices recommend applying the network policies to all pods. It is risky for teams to design network policies but accidentally forget details and leave it unsecured. A good practice here is to *default deny policy* [25]. Consequently, all traffic is blocked until new policies are created for the pod.

Network policy construction is supported natively by using Kubernetes network policies. They can also be created using a CNI plugin such as Cilium and Calico. Daniele et al. developed automatic network generation [36]. Now, there are more unexplored tools invented to support generating policies, such as Helm-ET and NeuVector [37, 38]. However, designing network policies is still challenging for teams and requires significant effort. After policy generation, policy verification becomes a concern. A tool such as Kano, which utilizes formal verification models, is available for this purpose [39]. Later, Shunbin et al. proposed a faster method for network policy verification that uses bitwise compression. [40].

3.1.6 Namespace Resource Quotas

The *namespace resource quota* is a mechanism used to control resource limits available to all objects in namespace scope [16]. It is recommended by Hamed et al. that different teams work on the same cluster [41]. The resource quota can manage maximum resource requests and limits for CPU and memory, the sum of storage requests, and finally, the number of different objects in a namespace. It uses the ResourceQuota object, which is illustrated in an example 1 below:

```
apiVersion: v1
  kind: ResourceQuota
  metadata:
    name: compute-resources-demo
  spec:
    hard:
      requests.cpu: "1"
      requests.memory: "1Gi"
      limits.cpu: "2"
      limits.memory: "2Gi"
      requests.nvidia.com/gpu: 4
```

Listing 1: ResourceQuota configuration for Kubernetes [16]

The example of resource quota 1 apply theses configurations into a namespace:

- request.cpu: Limits the total sum of CPU requests across all pods in the namespace to 1 CPU core [16].
- request.memory: Limits the total sum of memory requests across all pods to 1 gigabyte [16].
- limit.cpu: Limits the total sum of CPU limits across all pods to 2 CPU cores. Limits define the maximum CPU a pod can use. [16].
- limit.memory: Limits the total sum of memory limits across all pods to 2 gigabyte. [16].
- requests.nvidia.com/gpu: Limits the total number of NVIDIA GPUs requested by pods in the namespace to 4. This is specific to environments with NVIDIA GPU support configured. [16].

3.1.7 Dynamic Admission Control

Dynamic Admission Control is a component in Kubernetes that intercepts requests to the API server. It has an advanced function to allow customized configurations without impact on the kube-apiserver [3]. Dynamic admission control, implemented via admission webhooks, significantly strengthens Kubernetes security by providing a flexible and proactive mechanism for enforcing custom security policies at the point of resource creation or modification. It can safeguard cluster resources both during runtime and at deployment. In the runtime phase, the admission controller prevents unauthorized access and configuration changes. Unlike built-in controllers, it can be customized to address specific security threats [42]. During pod deployment, the dynamic admission controller can reject any deployments that violate established policies. It can be integrated into the CI/CD pipeline, aligning with modern software development processes and integrating with image scanning tools to prevent the deployment of vulnerable images [43].

3.2 Workload and Pod Level Security

Workloads or pods are the most fine-grained means to run the main business logic of the application. Therefore, they should be taken care of. This subsection discusses security issues related to configuration to protect a single workload running in the cluster.

3.2.1 Pod Security Standards and Pod Security Admission

Securing configurations in Kubernetes workload presents significant challenges. An improperly configured pod may be vulnerable to exploitation, which could potentially compromise the integrity of the entire cluster. Pod Security Standards (PSSs) make the security configuration painless and efficient. PSSs are Kubernetes policies for pods to ensure that all pods in the cluster adhere to consistent security baselines, minimizing vulnerabilities while maintaining operational flexibility with different levels. The PSA enforces these policies. Three levels of Pod Security Standards are:

- **Privileged:** Open policy providing the widest level of permissions. It permits some privilege escalations. [3].
- **Baseline:** Policy that minimally restricts and controls some privilege escalations. It permits the default configuration for Pods with minimal specifications. [3].
- **Restricted:** The most restricted one, using hardening best practices for Pods [3].

Pod Security Admission (PSA) is a built-in admission controller embedded in the kube-apiserver that enforces the PSS. If a request violates the policies defined in the PSS, PSA will react in 3 different modes:

- **enforce:** Policy violations result in the rejection of the pod. [3].

- **audit:** Policy violations lead to an audit annotation being added to the event recorded in the audit log, while they are still permitted. [3].
- **warn:** Policy violations result in a warning for the user. However, they are generally permitted. [3].

PSS and PSA work together as a two-part system. PSS provides the policy framework, while the PSA module operates it through namespace labels and real-time validation of pod creation and update periods. This duo is Kubernetes' primary pod security mechanism, ensuring clusters are secure, flexible, and easy to manage. An admin labels a namespace with a PSS level. When a pod is created, PSA reads the namespace label, fetches the corresponding PSS policy, and validates the pod's spec. Depending on the configured policies and PSA mode set, PSA may reject, log data, or trigger an alert.

3.2.2 Deploy Honeypot Workload and Intrusion Detection Systems

Canary deployment is a strategy to deploy multiple versions of an application and route a small amount of traffic from a small user subset to the new pod versions before fully rolling out. This technique can be applied in the Kubernetes environment. Fake workloads and services are deployed in the cluster. These resources act as a honeypot and are intentionally exposed to vulnerabilities in the cluster to monitor and notify the team of intrusions and attackers. Attackers can not differentiate between real and fake ones. Anomalous patterns will be monitored and detected by IDS [25]. After the attack actions are detected, the attack is traced back to the initial vector and alerted to the team [25]. The setup for this approach is presented in Figure 5. When an attacker sends malicious traffic to the Kubernetes cluster, they are likely to target the vulnerable pod, which is actually the honeypot. This diverts the attacker away from the privileged pod, which contains the actual sensitive data or critical workloads.

Other studies show that honeypot is an effective weapon in Kubernetes security. For example, Noah et al. built a honey pod system using Docker, Kubernetes, and workflow tools such as Apache Airflow, Spinnaker, and Argo for 3 months to uncover the risk associated with a Kubernetes system [44]. They received 191 attacks and categorized 4 main types of attack on their cluster, including exploiting the exposed service, establishing persistence on the hijacked system, cryptocurrency mining, checking for updates, or continuing to spread. In detail, the cluster was attacked by 3 big groups and others by diverse methods. The activities on containers are installing a rootkit, container escape, establishing persistence, disabling host defenses, installing crypto-miners and launching an unprivileged container. In the Kubernetes layers, they initially exploited the API server, kubelet, and the exposed endpoints. A similar research from Chakshu et al. is conducted on GKE honey pot system in 2 weeks to collect attack data from 200 IP addresses [45]. They recorded 11,500 brute force attempts, and the successful ones have similar behavior to mining cryptocurrency using the cluster's CPUs. All these research studies reveal beneficial information about the common attack vectors of Kubernetes in real-world scenarios and the attackers' behavior.

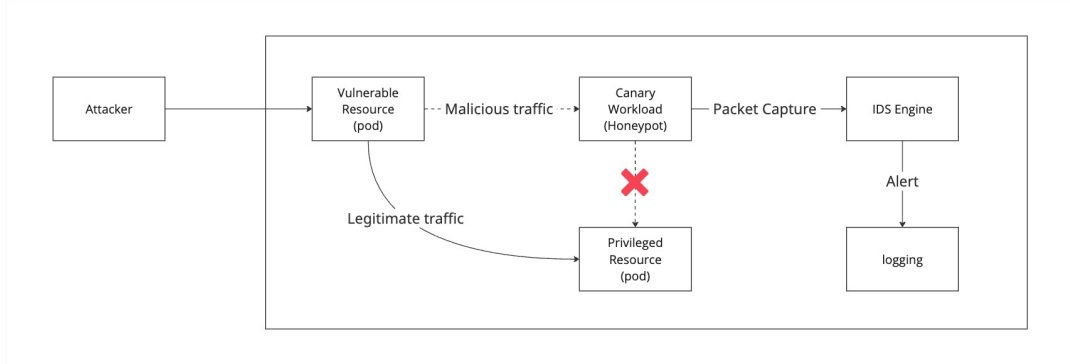


Figure 5: Kubernetes honeypot [25]

3.2.3 Static Analysis of YAML file and configuration

YAML files are created during the development process. These files can be created by developers without much regard for security practices, or they simply make it wrong. A YAML file configuration can be extremely complicated with many properties and indentation without a clear guide and standard, and they do not know how to securely generate YAML files for Kubernetes objects such as workloads and services. Wrong configurations in YAML files again exposed the attack surface and led to unpredictable consequences. Akond et al. identify main categories of misconfiguration categories with correlated Kubernetes objects, collected in Table 2 from [46]:

Kubernetes Object	Misconfiguration
Load Balancing for Meshes	Insecure HTTP
Secret	Hard-coded secret
Pod	Absent securityContext, Absent Resource Limit, Activation of hostPID, Activation of hostIPC, Activation of hostNetwork, Escalated Privileges for Child Container Processes, Insecure HTTP
Stateful Applications	Privileged securityContext, Activation of hostNetwork, Capability Misuse
Process Execution	Activation of hostPID, Activation of hostIPC, Activation of hostNetwork, Docker Socket Mounting
Traffic Routing	Insecure HTTP

Table 2: Kubernetes misconfigurations by object type

Aside from the listed categories, another study conducted on 287 open-source applications by Bufalino et al. identifies misconfigurations mainly related to the service labels and network configuration collision [47]. To mitigate this type of misconfiguration, it is recommended to use static scanning and analyze YAML files. The static tools, such as *kubesecc* or *kubetest* can detect demanding flags required for security [22].

3.2.4 Resource Management for Pods

According to Eunsook et al., CPU-intensive pods lead to degradation of cluster performance [48]. The same research shows a decrease in half of the CPU utilization. When a new pod is created, the kube-scheduler monitors the resources of the worker nodes and tries to assign the new pod to one of them. It also points to another root cause of high CPU usage, which is that network workload consumes and raises a research question for further resource management of Kubernetes. In another research, Tan et al. created the AlloX scheduler using Go and Python to optimize the dynamic fair allocation of CPU and GPU resources on a Kubernetes cluster for machine learning jobs [49]. Zhang et al. designed a scheduling algorithm, CloudSim, which is claimed to be better than the original scheduling algorithm and reduces cost [50]. For developers at the moment, resource requests and limits allow a simpler and default way of allocation and limitation of resources to Kubernetes objects. To do it, it needs to compute resources requested for the pod, and this can be configured in the workload YAML manifest or a specific pod object. Resource management at this level allocates and limits resources in a single Kubernetes object. Configuration can be determined for CPU and memory [16]. For instance, the listing 12 is the body of a manifest file that indicates request resource limitation with half cores and 300 megabytes. However, according to [41], it is dangerous to start the pod without an accurate resource limit specification. If the total resource limits exceed the total capacity of a node, and each node attempts to use resources up to its specified limit, the Linux OS kernel will ignore these limits. It will instead allocate only the requested resources or the resource allocation will be adjusted according to the ranges specified by Kubernetes. To crack this issue, Hamed et al. proposed a model to equitably calculate and allocate resource limits among the pods [41].

3.3 Container Security

Developers typically work with containers and images. Their responsibilities include building containers for applications, and security is a critical aspect that needs attention. However, conducting the security tasks is challenging. A research by Dongmin et al. explores misconfigurations in 2,680 exposed Docker Registry HTTP 411 APIs and 844 Docker daemon components from 14 large cloud providers. The same research also discovered 36 misconfigured servers in the Fortune 500 and 21 in diverse government servers [51]. This critical hardening process should begin during the image creation process with the appropriate configurations to limit exposure, followed by scanning at different stages. This section will detail how to create secure images and containers for development teams.

3.3.1 Build Secure Image

In the Kubernetes environment, applications are deployed using container images, which are typically stored in registries such as Docker Hub or private repositories. The image defines application code, dependencies, and runtime environment. It is

critical that the built image be secured to remove the attack surface. To address this, several practices are taken when building a secure image for Kubernetes [25, 22]:

- AppArmor is a Linux security module that supports configuring and managing container permissions on files and resources of container processes. Hui et al. created *Kub-Sec*, an automatic AppArmor policy generator for the whole distributed Kubernetes cluster. [52].
- It is recommended to use a minimal version of the trusted base image, such as an immutable Linux distribution, Alpine, or distroless images from trusted sources, to reduce the attack surface. The base images should only comprise runtime dependencies for the application. [25]
- In addition to minimizing image size, debug utilities such as shells, file system utilities can be removed [22]. This presents an attacker with the ability to escalate using common tools and reduces the attack surface.
- Kubernetes does not have image verification. This can be solved by using image signing and a Kubernetes admission controller. The admission controller can enforce a policy requiring image signatures and verify the image's validity.
- After deployment, many containers run as root users. An attacker can exploit and compromise the whole system. Creating non-privileged users can be run in the image Dockerfile configuration [25].
- Even images can be stored in both private and public registries, it should be stored in a private registry to control read and write permissions and grant read-only accounts for pulling images [22]

3.3.2 Image Scanning

Image scanning should be done at multiple stages of the application, including the build stage, the deploy stage, and the runtime stage. The scanner is usually integrated with the CI/CD pipeline. The scanned information includes OS packages, dependencies, file systems, passwords, certificates, and binaries. The scanner tries to collect data from the file system and match it with a vulnerability data source such as CVE [25]. Even if an image passes the static scanning and is deployed in the runtime stage, new critical vulnerabilities can be found and updated in the database [16]. There are many scanner tools available, including open-source and commercial ones, but choosing one should be considered based on privacy concerns. Sensitive data may be leaked from the image, then stored and misused.

Image Scanning in development phase

Automation for images scanning to detect vulnerabilities in the early stage is a best practice [22]. In the development stage, the image is first created and optionally pushed into the registry. The scanner later analyses image layers with existing vulnerabilities

in the database. The build pipeline should stop and reject an image that fails the scanning process [22].

Image Scanning in deployment phase

Even if image scanning is done during development, it can be activated in numerous stages, especially with the adoption of DevOps. New vulnerabilities can be found during deployment scanning, even if they pass the first scanning in development. It is the best practice to scan images in multiple stages of software development. The *ValidatingAdmissionWebhook* is an admission controller designed to support validating requests to the kube-apiserver. It checks images belonging to workloads that are ready for deployment. If the images do not meet the defined security scanning requirements, the workload will not be accepted.

Sysdig has developed an open-source project called *Anchore Engine* to facilitate this process. Here's how it works in conjunction with the *ValidatingAdmissionWebhook* [16]:

- A request to create a workload is transmitted to the kube-apiserver and then forwarded to *ValidatingAdmissionWebhook*. It will later be processed by the Sysdig scanning tool.
- The scanning tool evaluates the image and returns the results to the kube-apiserver.
- Based on the scanning results, the kube-apiserver makes the final decision to either approve or reject the workload creation request.
- In summary, the decision to reject or approve the workload lies with the kube-apiserver, guided by the results from the scanning tool.

Image Scanning at the runtime stage

After being deployed to the cluster, images are frequently re-scanned by the scanner to check for new vulnerabilities in the database. It is recommended to trigger the scanning process when new data is updated to NVD [16, 25]. In case a new vulnerability is detected, the failed scan can alert the operator to take action quickly.

3.3.3 Container Threat Analysis

Beyond traditional image scanning, the more advanced and new security practice for the images is container threat analysis. This is a sandbox-based solution that utilizes machine learning and smart algorithms to detect vulnerabilities in running containers. It tracks the network traffic, processes, system calls, memory, and other behaviors. Furthermore, file systems are checked to detect malware and threats [25]. This method is gaining popularity with solutions delivered in the market, such as Aqua Security's Dynamic Threat Analysis, Google's Container Threat Detection, and Sysdig Falco [53, 54, 55].

3.3.4 LimitRanger

LimitRanger is a built-in admission controller, and LimitRange is an object in Kubernetes. LimitRange is responsible for setting minimum, maximum, and default resource limits (like CPU and memory) for containers and pods within a namespace. LimitRanger then verifies and rejects any request that violates the constraints [25] in LimitRange. Violated request will receive a 403 status code response [16]. The example 13 configuration below shows the setting of the minimum of 0.05 CPU cores, maximum of 0.5 CPU cores, and memory limit of 512 mebibytes and minimum 50 mebibytes of memory for the namespace *demo*.

3.4 Monitoring, Alerting and Auditing

When a Kubernetes cluster is running, it is the perfect target for an attack. Therefore should be strategies to protect it during the running time. Most of the configurations mentioned before protect the cluster from threats while it runs. Moreover, it can be pushed further with protection during runtime. This section discusses the three most important strategies in the runtime phase of the Kubernetes cluster, including monitoring the system, alerting errors and audit logs.

Monitoring

Monitoring in the Kubernetes system involves measurements and metrics used to collect real-time data about cluster health and resource utilization, to alert [25]. Data used for monitoring are pods logs, network logs, application flow logs, and metrics that should be monitored are connection per second, data per second, request per second, CPU, and memory usage. This enables proactive detection of bottlenecks, failed deployments, and scaling needs, ensuring optimal operations and quick troubleshooting. In addition to security, monitoring helps detect any degradation in the quality of service, which is presented in [56].

Note that the nature of Kubernetes is a distributed deployment. The monitor has to be conducted on a group of nodes. Combined with the complexity of Kubernetes architecture with many pods, network, and this makes monitoring a resource-intensive task and requires necessary configuration to balance the visibility and expansiveness [25].

Monitoring can be implemented using a third-party tool such as Prometheus and Grafana. Kubernetes provides metrics that are used for collecting data about pod and cluster-wide scope by Prometheus [25]. Grafana supports advanced functions for visualizing data in charts and creating valuable observability for teams. Machine learning is also really promising now as a monitoring mechanism. Josue conducted research on machine learning for anomaly detection in Kubernetes, yielding effective results for alerting outliers and pledging broader applications in the future [57]

Alerting

Alerting is the action of transforming monitoring data into actionable notifications for critical events. Kubernetes does not support an alerting function [5]. Regardless of this weakness, Prometheus offers Alertmanager, an alerting component in the ecosystem. Alertmanager routes the received alert to the appropriate notification channels. Alerting everything is not logical. Teams can configure rules to effectively alert and prioritize high-severity issues with clear severity tiers (e.g., critical, warning) to avoid notification fatigue [22]. Example cases proposed by Kubernetes project contributors in 2021 are:

- Modification of Pod's securityContext [58].
- Admission controller configuration updates [58].
- Accessing specific exposed files or URLs. [58].

Auditing

Auditing is a useful feature offered by the API server. It records sequences of activities and who accessed the resources. Auditing answers questions:

- What and when something happened [59]?
- Who initiated it on what [59]?
- Where was it initiated and going to [59]?

Audit rules are confined by audit policy, a Kubernetes object. It determines which event and what data are recorded. The audit policy is sophisticated, and the rule levels include:

- None: Do not record any events that comply with this rule [59].
- Metadata: Only events with metadata are logged [59].
- Request: Log events along with the request body and metadata. [59].
- RequestResponse: Log events along with the request metadata, body, and response. [59].

Using auditing enables teams to monitor and record malicious activities within the cluster. Auditing is essential for security and troubleshooting, as it offers visibility into cluster operations and helps detect unauthorized or suspicious activities.

3.5 Compliance Check

Compliance validation provides systematic verification that the Kubernetes system adheres to organizational policies and industry security standards. This process involves continuous auditing of cluster configurations against established benchmarks to identify deviations from security baselines. For Kubernetes environments, the *CIS Kubernetes Benchmark* released by the Centre for Internet Security and the *Kubernetes Hardening Guide* by the National Security Agency are 2 ongoing industry standards. Adapting to these documents is also a best practice that needs to be done during the development process. For a project using Kubernetes as an orchestration system, there could be a compliance team that is responsible for ensuring compliance checks are adopted by the team and the software [25]. Therefore, following the shift left security principle, using these compliance documents during the development stage is an excellent choice to remove security risks in the early stage.

3.5.1 CIS Security Benchmark

The CIS Kubernetes Benchmark [60], developed by the Centre for Internet Security, delivers prescriptive configuration guidelines for hardening Kubernetes clusters. It gives security recommendations for the direct configuration of Kubernetes resource components in the worker nodes and control plane. Structured around five critical assessment domains, it addresses:

- Control plane elements with API server, scheduler, controller manager [60]
- Etcd configuration [60]
- Control plane configuration [60]
- Worker node components (kubelet and kube-proxy) [60]
- Kubernetes policy enforcement [60]

All of these components are inherently part of the cluster, and any compromise can lead to the entire cluster being attacked. The checklist provided by the CIS Benchmark offers recommendations for hardening the resource components in the Kubernetes worker nodes and control plane. By using this configuration, teams have strong security settings on the available components in the Kubernetes architecture. Each item in the checklist returns details about the default configuration value, rationale, impact, audit, and remediation [60].

3.5.2 Kubernetes Hardening Guide

In contrast to the concentration in Kubernetes control plane and worker node resources. The Kubernetes Hardening Guide document [5], developed by the NSA and CISA, spreads the checklist across all practices of Kubernetes. It encompasses a broad range of Kubernetes practices, including pod security, network configuration, authentication, auditing, and vulnerability management. This broad scope ensures that organizations

can secure all aspects of their clusters, from workload deployment to runtime protection, making it a valuable industry document. The guide should be adopted logically during the development process using the structured checklist of actionable recommendations, designed to mitigate three primary sources of compromise including supply chain attacks, malicious threat actors, and insider threats

3.5.3 Related Work in Compliance Check

Compliance in cloud computing is a complicated issue that requires extensive research by developer teams and organizations. Amit et al. have conducted a comprehensive study to show compliance models of 13 categories related to risk including:

- "Data encryption, key management" [61]
- "Media protection" [61]
- "Identification, authentication and authorization" [61]
- "Virtualization and resource abstraction" [61]
- "Portability and interoperability" [61]
- "Application security" [61]
- "Security risk assessment and management" [61]
- "Privacy, electronic discovery and other legal issues" [61]
- "Contingency planning" [61]
- "Data center operations, maintenance" [61]
- "Incident response" [61]
- "Compliance, audit and accountability" [61]
- "Awareness and training" [61]

In the cloud industry, cloud organizations have their own compliance solution. First, Google Cloud Security Command Center is a centralized platform to manage risk in Google [62]. It can monitor compliance, detect violations, and enforce compliance. Similarly, other providers have their own solutions, such as AWS Config and Azure Policy, with similar functionalities in compliance checks [63, 64]. Prisma Cloud provides a solution of API-based compliance management service [65]. Virtualization organizations Red Hat OpenShift and VMware also have their compliance enforcement solution, such as OpenShift Container Platform and VMware Aria [66, 67]. They work for compliance enforcement and checking in containers and infrastructure. All these listed solutions by the organizations provide support for runtime compliance checking and the configuration of custom rules [68].

In the academic field, Many studies propose effective solutions for the compliance enforcement process. Mozhdeh et al. introduced a machine learning model that uses network traffic data to detect compliance violations of privacy policies with more than 86% accuracy [69]. Mehdi et al. proposed a real-time detection and recommendation on HIPAA compliance model [70]. Ghareeb et al. excellently provide a new compliance management solution for infrastructure as code [68]. Although there are many documents about compliance for cloud systems, the available document specifically for Kubernetes is still limited in both research and industry at the moment.

3.6 Research Gaps

Security practices have been improved in each version by the Kubernetes team and the security team. New vulnerabilities have been reported and recorded, and new tools used to harden Kubernetes clusters have been invented in the last few years. However, these works have also revealed some challenges and gaps:

- There is no default secure configuration. The main document page from the Kubernetes team also provides configuration tips with simple and minimal configuration to reduce error occurrence [3]. This increases the attack vector for inexperienced developers.
- There is no systematic approach for security when building and deploying Kubernetes applications. There are guidelines such as the Kubernetes Hardening Guide by the National Security Agency, the CIS Kubernetes Benchmark, and the main Kubernetes security document [5, 60, 3]. However, when applying for a software product, these guides create overhead and may slow down the development process. There is a need for clear instructions on how to secure a cluster for developers and system administrators.
- A complex architecture of Kubernetes makes it extremely hard to build a comprehensive supply chain security. All the mentioned guides cannot propose a friendly end-to-end supply chain security approach for a development process. There should be a method to build both efficient and secure software development.

The above research gaps highlight a need for a different security approach for developers and system administrators. This thesis primarily addresses these gaps and creates a new formal approach for building a secure Kubernetes cluster. This approach needed to be capable of being used by developers in the development process in general cases. There are different cloud Kubernetes services such as GKE, EKS, AKS, IKS, ACK, and on-premises environments. The new high-level approach should work for all cases. In addition, this approach needs to translate high-level recommendations to something that users and developers can easily apply to the system without slowing down the release cycles and collaboration process of teams.

4 Methodology

The primary goal for a new method is to provide a comprehensive approach for Kubernetes security. This framework's target is to cover the current gaps in the security of the Kubernetes system and other frameworks. This section is structured to introduce the ideas and instructions to apply, and how the proposed method bridges the current gaps in Kubernetes security. Section 4.1 discusses the overview of a new method. Section 4.2 explores the details of the main methods and displays their structure. Section 4.3 finally shows how it can be applied by developers in a real system.

4.1 Methodology Overview

Some Kubernetes security frameworks have been introduced in [Security Frameworks](#), including the 4C's Model, NIST SP 800-190, and Kubernetes Hardening Guidance. However, these frameworks just give a list of best practices and recommendations, and it is still far away from a comprehensive approach with fine-grained instructions for a Kubernetes system. This section introduces a new approach for using matrix-based Kubernetes security. An initial version of this matrix security is shown in [Table 3](#). Kubernetes clusters and Docker containers are both included in the scope of this new approach, which is a difference from the frameworks introduced earlier. The scope is limited on these technologies because they are the standard industrial ones at the moment. There are other alternatives used in the market for containerisation and orchestration, such as Docker Swarm and OpenShift. However, due to the difference in the nature of technology, this method is not guaranteed to be applied in those cases.

Another difference with other approaches is that this matrix is not just a siloed guidance with a list of practices. The matrix is inspired mainly by using ideas from the shift-left security and defense-in-depth principles introduced in section [Security Frameworks](#). Martin et al. recommend implementing a defense-in-depth strategy for software applications [71]. Consequently, this approach is an effective way to defend different layers of Kubernetes systems. Research by Zlatan et al. has the same idea to conduct defense in depth in different layers as a main strategy to secure Kubernetes. They split the layers into components into control plane, node security, image and workload, access management, observability and auditing [72]. In contrast, this thesis work divides layers by architectural structure of a cluster from the container to the pod and cluster-wide level. It is crucial to apply the methodology across different layers cause Garsha claimed that Kubernetes has a layered architecture [35]. Each layer has its own security requirements and settings. This layered architecture requires defense in depth, which is both effective and desirable, as long as each layer is properly secured [35]. By using a 2D matrix-based approach, it can be ensured to cover security practices at each stage of the development process. This matrix supports both developers and cluster administrators to keep track of the security status for the whole cluster. It simplifies the complex structure of Kubernetes and its attack vector. That's why this is more comprehensive and better than other frameworks.

	Development Stage	Deployment Stage	Runtime Stage
Cluster Layer	Role-based Access Control Authentication and Authorization Secrets Management Namespace Boundaries Namespace Resource Quotas	Dynamic Admission Control	Monitoring, Alerting and Auditing
Workload/Pod Layer	Static Analysis of YAML file and configuration Network Policies LimitRanger Resource Management for Pods	Pod Security Standards and Pod Security Admission	Pod Security Standards and Pod Security Admission, Deploy Honeypot Workload and Intrusion Detection Systems, Monitoring, Alerting and Auditing
Resources Layers	CIS Security Benchmark		
Container Layer	Build Secure Image Image Scanning in development phase LimitRanger	Image Scanning in deployment phase	Image Scanning at the runtime stage Container Threat Analysis

Table 3: The proposed matrix of Kubernetes security

4.2 Matrix Design

The Kubernetes matrix is designed in 2 dimensions. The columns include different stages of a software development process. In this minimal and introductory version, there are 3 phases defined, including *Development Stage*, *Deployment Stage*, *Runtime Stage*. These are inspired by the DevOps life cycle and closely related to this cycle in 2.3. On the other hand, the rows cover different layers of a Kubernetes cluster, including *Cluster Layer*, *Workload Layer*, *Resources Layer*, *Container Layer*.

4.2.1 Layer Definition

From the lowest layer, the container layer focuses on the fundamental building block of Kubernetes's workload, which is the Docker image and the Docker container. At this layer, security practices are applied to make sure attack vectors are cut down from the containers running application services. This includes configurations of Docker images and libraries installed in the container environment.

Resource Layer represents resources running inside a Kubernetes cluster. This layer focuses on practices that protect Kubernetes resources in the control plane and worker nodes. The resources in the control plane are the API server, etcd key-value database, and scheduler. The worker node includes kubelet and kubeproxy. These components are the core of Kubernetes architecture and have been illustrated in section 2.2.1. Each component is really important, and they have different special permissions

to operate the entire cluster. Node components maintain pods and applications running, while the control plane controls the whole cluster. In this layer, security is assured by applying CIS Benchmarks, a set of configurations.

Workload Layer includes practices that work for a group of applications running business logic inside pods. The main practices in this layer harden the pods and applications with static scanning to detect vulnerabilities. In addition, the configuration covers resource management.

The highest layer, which is the Cluster Layer, comprises protections for the whole system at the cluster range and configuration impact on different namespaces. The practices for this layer such as RBAC, secrets management, and namespace resource quotas, have the highest impact scope.

4.2.2 Stage Definition

The proposed matrix is inspired by the DevOps life cycle at [2.3](#) and organizes Kubernetes security practices into three core stages. These stages provide a structured approach to securing Kubernetes environments while allowing flexibility for adaptation based on project-specific workflows.

- Build stage: software is designed with requirements and developed.
- Deployment Stage: software is built into the artifact and deployed.
- Runtime Stage: container and cluster are running and maintained in the cluster.

While these 3 stages provide functional structure, the development team may adjust the matrix to fix their need. There are possible added stages for a workflow:

- Pre-Build Stage: A team can integrate preliminary steps for security, such as threat modelling and security planning, and document preparation for suitable cloud providers.
- Sub-stages before release: Before the final release, there can be multiple stages for testing in a pre-production environment. In this stage, practices can be applied to validate network policies and fine-tune RBAC rules before releasing to production.
- Termination Stage: A cluster can include many jobs and cron jobs that request persistent volumes during task running. This supports cleaning up processes to clear data remains and frees up space.

These variations accommodate diverse workflows, such as legacy system integrations or multi-cloud deployments, without compromising security. While DevSecOps pipelines automate security across the software development lifecycle, this framework differs in scope and flexibility. This matrix allows dev teams to choose solutions, security tools, and implementations that best fit their environment.

4.2.3 Item Definition

The proposed Kubernetes security matrix provides a systematic framework for addressing risks across the entire lifecycle of a Kubernetes environment. Each cell in the matrix contains multiple security measures designed for specific Kubernetes components, such as the control plane, pods/workloads, or containers, at a defined stage of development. Arrangement of items is presented in [Table 3](#). Each item in the matrix is described in detail in [section 3](#). The item reference would describe in detail instructions on how to conduct the practice. This approach not only provides a high-level blueprint but also provides detailed instructions to implement the configuration.

In short, the matrix provides a high-level, adaptable blueprint for securing Kubernetes. Each cell references detailed instructions for engineers, such as configuring Pod Security Admission (PSA) for Workloads, integrating Trivy into image scanning, etc. This end-to-end mitigation demonstrates how the matrix ensures risks are fully addressed.

4.3 How to Apply When Building And Releasing Software

The matrix serves as both a strategic guide and a practical checklist for securing Kubernetes environments. By aligning security practices with software development stages and architectural layers, developer teams can systematically mitigate risks while maintaining agility. However, it can be challenging to apply this approach to a real software engineer's standard process. In real-life scenarios, developer teams work in a linear order with tasks followed by the next tasks. There must be a method to convert the 2D approach into a linear order of configuration. This can possibly be done by applying the two methods below: layer by layer and stage by stage.

4.3.1 Layer By Layer

In a layer-by-layer implementation, secure configurations are applied and tested for each layer. In a simple term, the security practices are conducted from the bottom to the upper layers. First, an application with multiple services and clusters can be built, deployed, and run. Second, image and container hardening are involved to ensure security at the first layer for containers. Then, configuration for middle layers, which are components in the worker nodes, pods, and the control plane, is implemented. Finally, the system is tested with practice from the top layer with a configuration correlated to the whole cluster's impacts.

It is not mandatory to configure layers in the above order. Each layer target is isolated, and the configuration in each layer does not impact the others. However, after each layer configuration, to make sure the functional requirements of the project work rationally without broken features, the whole application flow should be end-to-end tested. This method requires many iterations of configuration and testing, but it is more friendly adapted and conducted. Any interrupted setting up could be easily found and fixed.

4.3.2 Stage By Stage

Different from the layer-by-layer implementation, the stage-by-stage method requires teams to come up with the configuration across the whole cluster. This is slightly similar to the *Waterfall Model*, a traditional software development method introduced in 1970. All requirements and configurations are analyzed and defined carefully. This makes the matrix implementation seem to be short for a project, but the workload for each stage is enormous, especially the Development Stage. Moreover, it is extremely complicated to troubleshoot errors.

4.3.3 Agile Implementation

Compare the two implementations above; each has its pros and cons. In modern software development philosophy, iterative and incremental approaches are foundational. Iterative elements allow the developer team to refine features based on feedback, and incremental elements ensure stable progress.

Incremental principles support the team in releasing the features quickly. It is now common for organizations to release prototypes or minimum viable products (MVPs) rapidly to gather feedback. From the feedback, features are updated, upgraded, and added iteratively. Combined with DevOps, new features are continuously integrated and delivered (CI/CD). To adapt to modern software development methodologies and to make expanding the software with new features smooth, an iterative approach, which is layer by layer, can be applied to the proposed matrix. A simple way to execute the matrix for an initial insecure cluster is presented in section [4.3.1](#). Furthermore, when new features or services are added later, multiple iterations, which means multiple layers, can be applied to the new features. This makes the product delivered smoothly to end users without much interruption to the experience. This approach will be performed in the next section with a real case study implementation.

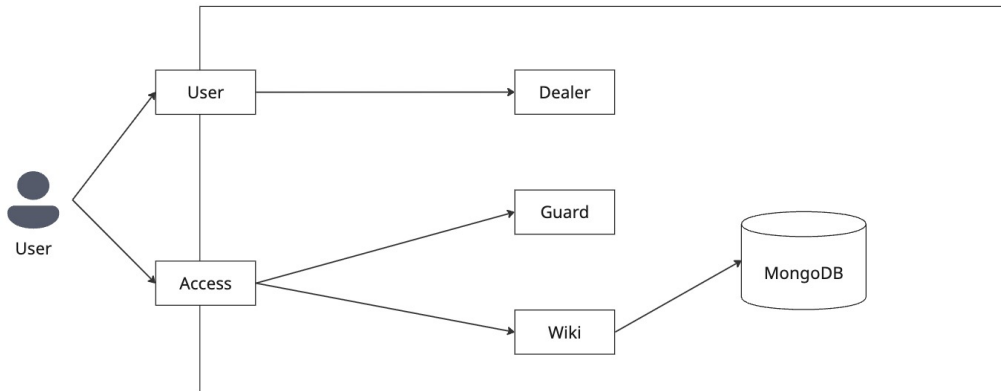


Figure 6: ARA2 architecture

5 Case Study And Result

In this section, a case study application is implemented to validate the effectiveness of a matrix-based security approach for hardening a Kubernetes cluster, addressing risks at the container, node, Pod, and cluster levels. The application overview and cluster environment are presented in section 5.1. Section 5.2 will describe in detail the configuration of container and image security. Section 5.3 discusses the practice of hardening components in the control plane and the worker node. Section 5.4 shows configurations for Pod, which include higher-level workloads such as StatefulSet, Job and Deployment. Finally, section 5.5 examines practices for cluster-wide configuration.

5.1 Application Overview

Anonymous Registration Anonymous Access Protocol (ARA2) is a distributed authentication system designed for Kubernetes environments. This system is a cryptographic flow that allows users to authenticate using cryptographic token exchanges between decentralised components and then access protected resources. The ARA2 application serves as an ideal examination for the security matrix due to its multi-component architecture, sensitive data and complex multiple operations. This system consists of the main custom services: *Dealer*, *Guard*, *Access*, *User*, *Wiki* and 2 dependencies: *Strimzi Kafka Operator* and *MongoDB*. Figure 6 show the architecture of the case study system. All services in ARA2 are deployed across multiple instances to create a distributed environment. All user requests are HTTP-based, and all connections between services in the cluster are made through the Strimzi Kafka Operator.

The general workflow of the system can be simplified into three main processes:

- System setup: Dealers generate a token and distribute it to Guards. Guards then generate a token from the received data.

- User Authentication: User identifies using the generated token to the Dealer via the User proxy.
- User Authorisation: User accesses through the Wiki service using a user token and an access key. The combination is sent to the Guard to validate permissions

5.1.1 Dealer

The Dealer service is the first crucial component in the protocol. It provides the access key token to the user. Given the requirement of anonymous and distributed authentication, the Dealer must remain a separate component, ensuring it cannot obtain more information than necessary to provide the access key. Additionally, since the Dealer's degree must remain secret, it operates as a multi-instance component, with each instance holding its own degree. The Dealer's main purpose in the setup stage is to generate its own polynomial, partition it into N shares and send it to the Guard. During the authentication stage, the Dealer must verify the identification credentials and reply with its token.

The life cycle of a Dealer comprises the following steps:

- Connect to the Kafka broker
- Subscribe to its topic
- Request for the topics of the Guards from the Kafka broker
- Check for the existence of a state file (which indicates a previously aborted execution).
- If the state file does not include the value of its polynomial or the state file does not exist, generate its own polynomial and save it in the state file.
- If the state file does not indicate that the token was already sent to the Guards, partition the token and send it to the Guards.
- Continuously for messages from the AuthManager, including the user's identification, and process the messages accordingly

5.1.2 Guard

The Guard component is crucial to the implementation of the protocol as it verifies user access to the resource. It functions as the Guard in the original protocol. Due to the requirement for anonymous and distributed authentication, the Guard must remain a separate component. It should not acquire more information than necessary to verify user authorization. Additionally, since the Guard's own computed degree must remain secret, it is a multi-instance component, with each instance maintaining its own degree. The Guard's two main functions are exchanging messages with Dealers during the setup phase to create its own key and use that key to verify users' requests to resources.

The life cycle of a Guard consists of the following steps:

- Connect to the Kafka broker
- Subscribe to Kafka topic
- Check for the existence of a state file (indicating a previously aborted execution)
- If the state file includes its own token, set its current token to the one stated in the file
- Continuously listen for messages from the Dealers containing their partitioned tokens, add them to its current token, and update the state file
- Continuously listen for messages from the Access containing the tokens of users attempting to access the resource

5.1.3 User

This service communicates with the Dealers through the Kafka queues. The main function of the User service is to authenticate users by forwarding messages between users and Dealers. This service is exposed as an HTTP API in the cluster. After users get the static authentication page from Access, they send a request to the User service API, and messages are finally forwarded to the Dealers.

The life cycle of a User service consists of:

- Connect to the Kafka broker.
- Subscribe to Kafka topic
- Expose HTTP API and forward requests to Dealer
- Listen for Dealers' replies and respond Dealer tokens to the user

5.1.4 Access

Access service allows users to fetch the authentication page in the first stage of the flow. After being authenticated by the User service and the Dealer. Users access the resources through Access again. The request would be authorised by a Guard at this stage before going to the Wiki Service.

The life cycle of Access includes:

- Connect to Kafka broker
- Subscribe to Kafka topic
- Expose and HTTP API to serve an HTML page
- Continuously listen for messages from the Guards and store them until all of the corresponding texts for a user are received.

5.1.5 Wiki

Wiki component allows users to perform read, create, and update operations on data which is stored in a MongoDB database. Users can only access this service and request data if they have permissions. Permissions are keys received from Dealers, and they are first verified by the Guard component. After the authorisation, the user can then access the persistent data in the cluster.

5.1.6 Environment Setup

ARA2 is deployed in the remote Aalto described in table [Hardware specifications](#), and the Kubernetes cluster is described in table [Cluster specifications](#).

Property	Details
Cluster Type	Self-managed on Aalto University VM
CPU	8
CPU Architecture	x86_64
RAM	16GB

Table 4: Hardware specifications

Component	Details
Nodes	1 control plane, 1 worker node
Kubernetes Distribution	Microk8s v1.32.3
Kubernetes Version	v1.32.5
CNI	Cilium v1.15.2
Node	k8s-scenario0.cs.aalto.fi
Operators	Prometheus, Strimzi Kafka

Table 5: Cluster specifications

In the case study, each layer is adapted from bottom to top. There is no correct order to configure the layer, but the bottom-up approach is developer-centric. However, it should be noted that any configuration at each layer can break the whole functionality of the system. For any order, it should be taken into account that each configuration of a layer should be applied, tested, and verified carefully before going to the next one. This supports the team to work in a linear order of security tasks, which covers both dimensions of the security matrix. Due to the limited capacity of the thesis, not all listed practices in the matrix are applied. The case study and the implementation below are used as proof for the possibility of this approach. Teams can flexibly remove or add any configuration depending on the use case, security requirements, and system complexity.

5.2 Container Layer

Practices

The container layer includes the attack surface from the images and containers running in the Pod. This section describes the technical methods chosen and applied to the case study. The main practices applied to the container layer include distroless image engagement and image scanning. In the case study application, all the initial images are simple Docker images that install the Nodejs dependencies, build the application, expose a port and start the application. [Listing 2](#) is the beginning image build, which is simply a Nodejs version 20 image that installs build and then runs the application. Image securing practices have already been introduced in [section 3.3.1](#) and now applied. To make it more protected, the image is recreated using Google distroless image in [Listing 3](#) with multiple stages build. This practice contains only the application and its runtime dependencies. Multi-stage builds make the usage of distroless images easier [\[73\]](#).

```
FROM node:20

# Set the working directory inside the container
WORKDIR /usr/src/app

# Copy package.json and package-lock.json to the working
# directory
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code to the working
# directory
COPY . .

# Build TypeScript code
RUN npm run build

# Command to run the application
CMD ["node", "./dist/access.js"]
```

Listing 2: Docker image for *Access* Pod

In the first stage, all the dependencies and libraries are installed. The build stage uses node:20 for installation and compilation. The whole application is then built into *build-env* artifact. In the second stage, only the necessary build-env artifacts are copied to the final distroless image on gcr.io/distroless/nodejs20-debian12, which is a minimal nodejs 20 runtime. Although *nonroot* is a more secure option for the image tag, *debug* tag is used here for later work in debugging and configuration of other layers. For the final release of the production, the non-root tag is a better option for security. The final image now runs the compiled JavaScript using only minimal dependencies without a shell or package manager.

Distroless image usage implies non-root user access by default, so the Docker container has only a limited set of permissions. This works without the configuration

of a non-privileged user in the Docker image file. Moreover, this case study applies the practices of using a private registry instead of public registries such as GitHub Container Registry or Docker Hub Registry. Aalto Harbor is an image registry to store and manage images and artefacts provided by Aalto University [74]. All built images are pushed and stored in Aalto Harbour [74]. Read and write permissions are granted only to the project admin and the thesis worker. Moreover, all images pushed into the private registry are scanned by the Trivy scanner adapter. Trivy is an open-source security scanner tool that finds vulnerabilities in CVE and misconfiguration across code repositories, binary artefacts, container images, and Kubernetes clusters [75]. The scanning results will be evaluated in the next section 5.2. All these practices combined have limited the well-known attack surface on Docker images and containers used in a typical Kubernetes cluster.

```
# Stage 1
FROM node:20 AS build-env
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2
FROM gcr.io/distroless/nodejs20-debian12:debug
COPY --from=build-env /usr/src/app /usr/src/app
WORKDIR /usr/src/app
CMD ["./dist/access.js"]
```

Listing 3: Distroless and multistage built Access Docker image

Evaluation

This section discusses the quantitative results of the distroless image and image scanning on the main services. The comparison of a number of vulnerabilities is shown in Table 6. Distroless image shows impressive improvement in this case, which altogether removes 100% all the critical, high and unknown severity vulnerabilities on application Docker Images. The remaining vulnerabilities are unfixable medium and low-level ones, which are temporarily ignored.

	Initial Image					Distroless Image				
	Critical	High	Med	Low	?	Critical	High	Med	Low	?
Access	5	137	508	660	6	0	0	1	15	0
Guard	5	137	508	660	6	0	0	1	15	0
User	5	137	508	660	6	0	0	1	15	0
Wiki	5	137	508	660	6	0	0	1	15	0
Dealer	5	137	508	660	6	0	0	1	15	0

Table 6: Vulnerabilities scan result of regular and distroless Docker image

?: Unknown Severity Vulnerabilities

Med: Medium Severity Vulnerabilities

The tables 7 and 8 describe the Docker image misconfiguration scanning result. The image scanning for configuration passes 24 out of 26. The remaining misconfiguration requires a *HEALTHCHECK* instruction, which is not critical in security and can be overlooked at the moment. The other asks for the *USER* command to create a non-privileged user for the container. However, with the support of distroless images, a non-root user is applied by default, and the current Images configuration is adequate.

	Severity	Description
AVD-DS-0002	High	Specify at least 1 USER command in Dockerfile with non-root user as argument
AVD-DS-0026	Low	Add HEALTHCHECK instruction in your Dockerfile

Table 7: Docker image misconfiguration description

	Number of Tests
Successes	24
Failed	2

Table 8: Docker image misconfiguration result

5.3 Resources Layers

Practices

The Resource Layer addresses critical attack surfaces in Kubernetes infrastructure components - etcd, API server, kubelet, and kube-proxy - which form the backbone of cluster operations. As discussed in section 3.5, vulnerabilities in these components can lead to cluster-wide compromises (e.g., CVE-2020-8554 kubelet exploits). MicroK8s provides a streamlined security solution through its *cis-hardening* addon, which implements the CIS Kubernetes Benchmark v1.24 recommendations through automated reconfiguration. The CIS configuration can be turned on and off using listing 4.

The verification of the CIS Benchmark on a cluster can be performed using a Go application called *Kube-bench* [76]. MicroK8s also supports the installation of kube-bench when the cis-hardening addon is enabled. To call the kube-bench, run the command in Listing 5. It checks the deployed cluster strictly using details in the CIS Benchmark document. However, it is important to apply CIS hardening actions to each individual node before it joins a cluster.

```
$ microk8s enable cis-hardening
$ microk8s disable cis-hardening
```

Listing 4: cis-hardening usage in Microk8s

```
$ sudo microk8s kube-bench
```

Listing 5: Run CIS benchmark

The MicroK8s CIS Benchmark Structure are organised as follows [60]:

- Check 1: Control plane security configuration
- Check 2: Etcd node configuration
- Check 3: Control plane configuration
- Check 4: Worker node security configuration
- Check 5: Kubernetes policies

The details of CIS recommendations, which contain description, remediation, and expected output, can be found in the main document at [60]. From the checklist above, the CIS list provides sufficient configuration details for the Resource Layers of the worker nodes and control plane. The ultimate goal is to cover the attack vector at these nodes and components running inside. Although Microk8s supports many settings and remediation lists, there are numerous manually required configurations in the checklist. These manual checks are not automatically set by enabling CIS hardening on Microk8s but need to be manually verified by the cluster admin. Lastly, kube-bench will test and generate a detailed report for each item in the list and the associated fixes. The test result will be discussed in the next section.

Evaluation

This section presents the validation results before and after CIS technical configuration settings in the setup cluster. Summary policies represent the Check 5 for Kubernetes Policies, and summary total shows the total configuration test across the entire cluster, including control plane, etcd, and worker nodes. Without the cis-hardening add-on enabled, the validation output 4 failed tests for policies and 25 failed tests in total. The add-on reduces 100% of failed tests after being enabled.

The cis-hardening add-on should be turned on in the Development Stage or the Deployment Stage to manually verify the Checks. To control and enhance security in the Runtime Stage, continuous kube-bench monitoring can be run. The implementation in this case study is straightforward. However, different cloud providers utilize various Kubernetes distributions, which may require distinct configurations. The comparison between enable and disable configurations is demonstrated in Table 9 and 10.

	Summary policies	Summary total
PASS	3	48
FAIL	4	25
WARN	28	50
INFO	0	0

Table 9: Test result on CIS-disabled cluster

	Summary policies	Summary total
PASS	0	83
FAIL	0	0
WARN	30	41
INFO	0	0

Table 10: Test result on CIS-enabled cluster

5.4 Workload Layers

Practices

Trivy scanning not only works for images, but it can also operate on YAML setting files. Trivy scanning performs excellently in detecting misconfigurations. There are analogous options such as *kubesecc* and *kubescan*, but Trivy was used since it covers other steps of the process. To protect Workloads running in the case study, this work tried scanning and fixing all the misconfigurations in the YAML files.

```
{
  "istio-injection":"enabled",
  "kubernetes.io/metadata.name":"default",
  "pod-security.kubernetes.io/audit":"restricted",
  "pod-security.kubernetes.io/warn":"restricted"
}
```

Listing 6: Pod security standard configuration

Along with the static analysis of YAML files, Pod Security Standard and Pod Security Admission are also applied. The PSS is enabled and impacts the whole namespace. The *default* namespace that contains most of the main Pods now has new labels added. The new PSS configuration is described in listing 6. The PSS profile is set as *Restricted*, the highest level of hardening with the most restrained practices. To avoid breaking the application, the PSA mode is set to *audit* and *warn*. This configuration triggers warnings and audit annotations to the violated records of the PSS profile, which will be illustrated in the next evaluation section.

Evaluation

This section discusses the result of static analysis and PSS configuration for Pods running in the default namespace. By adding new PSA labels to the default namespace, all Pods created which is not strictly aligned with the Restricted rules of PSS generate warning messages. For example, [Pod security standard violation](#) shows a warning when a new StatefulSet named *guard* is created. The warning asks for fixing the configuration in the *securityContext* property of the container.

```
Warning: would violate PodSecurity "restricted:latest":
  allowPrivilegeEscalation != false (container "guard" must set
    securityContext.allowPrivilegeEscalation=false),
  unrestricted capabilities (container "guard" must set
```

```

securityContext.capabilities.drop=["ALL"]), runAsNonRoot !=
true (pod or container "guard" must set securityContext.
runAsNonRoot=true), seccompProfile (pod or container "guard"
must set securityContext.seccompProfile.type to "
RuntimeDefault" or "Localhost")
statefulset.apps/guard created

```

Listing 7: Pod security standard violation

	Initial Misconfigurations	Improved Misconfigurations
deployment.yaml	85	35
mongo.yaml	17	14

Table 11: Pod's misconfiguration numbers

Using Trivy scanning for misconfigurations returns the same recommendation in the securityContext property of the container. Table 11 presents the count of misconfigurations found in YAML files during the initial project setup and after implementing fixes recommended by the scanning tool. The deployment.yaml file contains the main workloads described in 5.1, and the mongo.yaml is the setup for MongoDB Pods, Service and Volume. There are 85 misconfigurations related to core Pods in Section 5.1 and 17 misconfigurations for MongoDB yaml files. The report shows the recommended configuration for securityContext, which is the same as using PSS.

Both the PSS and misconfigurations scanning enhance the Pod configuration. However, it is not simple to fix all the configurations without trading off for the availability of Pods. The current PSS config and scanning require Restricted policies for YAML files, but not all fix the issue in the case study. For instance, by following the securityContext policy of these practices, all running Pods show *permission denied* error in logs and disrupt the Pods. To balance the trade-off between availability and security, there are still misconfigurations from the Aqua Vulnerabilities Database remaining in the YAML implementation. Listing 8 displays a working version that improves configuration while keeping the cluster functional. The error appears for the following scanning results and recommendations:

- securityContext.runAsNonRoot
- securityContext.runAsUser
- securityContext.runAsGroup
- securityContext.capabilities.drop: [ALL]

```

securityContext:
allowPrivilegeEscalation: false
seccompProfile:
  type: RuntimeDefault
readOnlyRootFilesystem: true

```

Listing 8: Harden configuration for Pod

5.5 Cluster Layer

Practices

The Cluster Layer has the most complex security composition. While container and workload focus only on the configuration of static files and the Resource Layer is supported by kube-bench and Microk8s, the Cluster Layer requires many manual, error-prone configurations for RBAC, network policies, and resource management. The main practices applied in the case study are RBAC, secret management, and network policies. Both are hard to confine without solid domain knowledge and system architecture. The YAML in 9 shows the RBAC configuration.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: config-manager
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: admin-config-manager
subjects:
- kind: User
  name: admin
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: config-manager
  apiGroup: rbac.authorization.k8s.io
```

Listing 9: RBAC admin role

Network policies are the hardest but the strongest security practice to operate and limit the lateral movement of Pods. All the beginning network policies are configured as shown in the listing 14, 15, 16, 17, 18, 19. To harden the network policies, *Default Deny* at 11 is added to stop explicitly not allowed traffic.

Secrets contain sensitive data and need to be hidden from exposure. The most straightforward option is encryption at rest, which makes access to the service without the key useless. The configuration 10 reveals the encryption at rest for the cluster's data in the case study. The current implementation asks for encryption for Secrets only. In this case, the encryption provider used is *secretbox*, which uses strong and fast encryption algorithms *XSalsa20* and *Poly1305*. A 34-byte key is randomly generated,

base64 encoded for encryption and stored locally on the same host. The data encryption requires the cluster to restart.

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources:
    - secrets
  providers:
    - secretbox:
        keys:
          - name: key1
            secret: KE+7Na10oxIk6IhRyTiDyg5elpBQEFoRgblH7Vg
              +M1o=
```

Listing 10: Secret management configuration [3]

Evaluation

There is no quantitative evaluation result for the configuration of the Cluster Layer. However, there are diverse aspects to discuss about these configurations.

The ideal practices for network policies are to deny by default. In the case study, denying default is really hard to debug and configure. In addition, it breaks the reliability of the initial version that works. The configuration 11 fixes it by applying deny default on each Pod's labels instead of the whole cluster. The issue with denying by default on the entire cluster is the compatibility with third-party add-ons, Operators. The case study uses Kafka Operator, Prometheus and scanning Operator, which do not have a unified label convention. The application is unavailable due to the default deny blocking network to and from Operator Pods. One way to fix it is to manually adjust labels on Operators, but this is challenging when more Operators are added to the cluster and later become unmanageable. Instead of that, the deny rules in the case study are applied for each Pod individually for the trade-off of availability. This reduces the attack surface in the application Pods but also requires a future improvement on network security for Kubernetes Operators.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-wiki
spec:
  podSelector:
    matchLabels:
      app: wiki

  policyTypes:
    - Ingress
    - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-mongo
```



```

spec:
  podSelector:
    matchLabels:
      app: mongo

  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-guard
spec:
  podSelector:
    matchLabels:
      app: guard

  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-access
spec:
  podSelector:
    matchLabels:
      app: access

  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-dealer
spec:
  podSelector:
    matchLabels:
      app: dealer

  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-user
spec:
  podSelector:
    matchLabels:
      app: user

```

```
policyTypes:  
- Ingress  
- Egress
```

Listing 11: Deny all network policy

The Secrets encryption at rest provides strong protection for data. Unfortunately, it is not complete protection for the cluster's data. There are further practices that should be applied. The current implementation stores the key in a host, which means it is not safe for the host to be compromised. Once the attacker accesses the host, they can later use the key to decrypt all Secrets stored.

In this implementation, the key is created once and fixed. However, this nature has disadvantages for security posture. A more complete protection requires using key rotation. The keys are often recreated for encryption and decryption of data. Even if a key or secrets are compromised, their validity is limited to a short span of time. It is also immune to brute force attacks. For most tasks in secrets management, HashiCorp Vault provides much more advanced solutions, but it has not yet been applied in the case study due to its rough architecture and limited capacity for the work.

6 Discussion

This section discusses the effectiveness of the previous case study work and limitations for future exploration. Section 6.1 will cover how the new approach works. In section 6.2, challenges during the implementation of the case study are illustrated. Finally, section 6.3 will show the limitations of the current approach, and this leads to future open research questions on how to improve this new approach.

6.1 Case Study Output Result

From the case study section, the implementation of matrix-based security demonstrated significant effectiveness in hardening the ARA2 Kubernetes system. By systematically applying layered security practices aligned with development stages, critical vulnerabilities were substantially reduced. Most impactfully, the adoption of distroless images eliminated 100% of critical and high-severity vulnerabilities across all main services and reduced the workload misconfiguration by 52%. The CIS Benchmark automation via Microk8s further validated cluster-wide configuration and reduced configuration failures from 25 to 0. All these quantitative gains confirm the framework’s capacity to mitigate risks for a case study cluster.

Qualitatively, the matrix layered approach provided clear prioritization, while iterative per-layer testing prevented systemic failures. However, the dependency on third-party Operators complicated cluster-wide policies, revealing ecosystem integration as a key challenge. The shift-left integration of practices like development stage image scanning also proved valuable for early vulnerability detection.

On the other hand, the most important discussion is the trade-off between security and availability during implementation. PSS restricted mode and recommended configuration cause the breaking of functionalities. Similarly, default-deny network policies disrupted Kafka Operator communications due to inconsistent labeling conventions.

This highlights the need for iterative experimentation to find out the optimal balance points with maximal security without functional corruption. In this work, the balance point is discovered for network policies and Pod configuration, but this point requires team effort and time investment.

6.2 Challenges

Implementing the proposed Kubernetes security framework presented several significant challenges that highlight the gap between theoretical models and practical deployment. First, a steep learning curve is an issue for the teams. There are many challenging issues, such as network policies requiring extensive documentation and testing without a proper, friendly tool. Moreover, it is not easy to exactly follow the best practice and make the application operate correctly. Practices such as denying all network policies do not work in the case study, causing the integration of uncontrolled Kubernetes Operators in the cluster. Operator Pods’ labels that are not well configured make it tough for teams to apply. In addition, each practice is defined in

the development life cycle, which requires huge effort for researching, integration, configuration, validation, and documentation. To overcome these challenges, teams need competency experts, which is a high cost for the organization.

Comprehensive testing across layers/stages created exponential validation overhead. Writing a unit and functional test case can be time-consuming. To cover different components of the cluster in a workflow, End-to-end (e2e) testing can be a solution. E2e testing addresses the challenge of testing large integrated systems and validates the whole workflow, but it may slow down the execution time [77]. It creates a complex test environment setup and test flakiness. These issues can lead to increased maintenance costs, slower feedback loops, and difficulty in debugging failures.

In modern software development, a cross-functional team setup is popular. To work together in the teams, tasks are assigned to members during the planning stage. A challenge of using this matrix is how configuration tasks can be distributed to team members to balance the coding and configuration sides without overhead and technical debt. Additionally, putting the matrix into use requires a cost in meeting hours increment, aligning the results, and learning and development. Moreover, in the context of Agile and Scrum software development, this requires closer collaboration between the Product Owner, the Scrum Master, and the Development Team. The duty of planning and dividing tasks is assigned to the Development Team, and this raises more costs for the organization.

The final challenge in this work is execution and adaptation on different cloud providers' services. There are many cloud providers in the market, with the biggest ones being Amazon Web Services, Microsoft Azure, and Google Cloud Platform, and the thesis aims to cover a provider-agnostic solution. To adapt to a detailed cloud environment, actions are required in researching and customization. Even the high-level details provide a comprehensive structure, efforts are demanded to transfer from theory to application on a specific cloud provider or on-premise environment, and require more volume of work. This work does not deliver instructions for migration between different cloud systems.

These challenges validate the framework's novelty, while existing solutions address isolated Kubernetes risks, this matrix approach uniquely simplifies and exposes systemic gaps in enterprise adoption. Future work will develop abstraction instructions to simplify environment-specific adaptations.

6.3 Limitation and Future Work

While the proposed security framework and the case study demonstrate the feasibility of Kubernetes hardening, several limitations merit consideration for further advancement. This section states limitations in this thesis work and provides future research directions to improve these limitations to provide the most complete guideline for the software industry.

Initially, the validation was constrained to small-scale clusters (1 control plane and 1 worker node), limiting insights into enterprise-grade environments. Using this simple environmental setup can't sufficiently demonstrate the distributed nature of Kubernetes and microservices, and can't show the power of resource management

practices in namespaces and Pods. Moreover, a bigger cluster means a huge number of roles, more network policies, more configuration practices, and a larger attack vector. The guidelines cover all aspects of the application; however, executing all of them may slow down the development process significantly. There should be a future investigation to find the correct balance point between security and development speed for teams. To strongly prove the theory and gather more accurate data, the case study should be conducted in a more complex system and a bigger environment cluster with multiple nodes.

Second, due to the limited capacity of the thesis works, not all the practices are applied, which makes it unsatisfactory to prove the theory. For example, in runtime monitoring, the work was not successful in scheduling the scanning of multiple layers of the cluster within the given time. This is a minor loss in the work, and there should be more practice applied to cover the whole matrix design. During the work, while the framework reduces theoretical attack vectors, its validation lacks specific adversary simulations to prove mitigation efficacy. Additional work on the attacking scenario can be done to illustrate the effectiveness of the thesis result for known threat patterns and real-world exploits, such as MITRE ATT&CK techniques and CVE-based attacks.

7 Conclusion

Kubernetes is a de facto technology for building scalable, resilient, and highly available systems. This also introduces critical security challenges for team members who interact with the cluster. These challenges come from the complex architecture of the Kubernetes cluster, which requires multiple practices and research, including misconfiguration analysis, secrets protection, runtime behavior monitoring, and so on. While existing guidelines, CIS Benchmark and NSA Hardening Guide, offer the best practices, they are far from a manageable systematic approach that could be integrated with the modern DevOps life cycle. Moreover, they still leave many attack vectors uncovered. This thesis work addresses the gap by introducing a robust and systematic approach as a 2-dimensional matrix. It divides the architectural design into containers, workload, control plane, and worker node components, and finally the whole cluster scope with 3 main stages shortlisted from the DevOps stages, including development, deployment, and runtime.

The new approach innovation comes from the combination of defense-in-depth and shift-left security principles. During the thesis case study implementation. Practices such as distroless containerization, image scanning, CIS automation via Microk8s, and network policies are enforced into layers at different stages to ensure vulnerabilities are detected, mitigated, and fixed systematically. The case study was validated on ARA2, an anonymous authentication system, where the result reached 100% elimination of critical and high severity vulnerabilities in the core containers. Despite challenges in fixing security recommendations, the iterative layer-by-layer implementation successfully delivered a healthy cluster, which proved the adaptability of modern Agile and DevOps flows into the hardening process. This later flexibly extends the development, security of new features and microservices added to the cluster.

Even though there are successes in the thesis work, there are limitations that remain. The case study was conducted on a single cluster with a control plane and a worker node only. Due to limited capacity, the work lacks efficient integration of runtime practices would give more insight about the runtime threats and generate more ideas to improve the current operation. All the limitations make the future work critical to extend the case study scale to match the nature of a distributed system, and perfectize the implementation of the whole matrix process. Academically, there are challenges pointed out during the implementation that raise many research questions about finding a balancing point of security and functionality, and also the question of security practice missing for third-party Operators running inside the cluster.

In conclusion, this thesis redefines Kubernetes processes, making fragmented guidelines into a unified, life cycle-integrated and systematic paradigm. In the near future, Kubernetes may evolve, and this matrix-based strategy is still a foundational blueprint that can better adapt to new architectures and adjustments.

References

- [1] *CNCF Annual Survey 2023*. CNCF. Apr. 9, 2024. URL: <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- [2] *What Is Docker?* IBM. URL: <https://www.ibm.com/think/topics/docker>.
- [3] *Overview*. Kubernetes. URL: <https://kubernetes.io/docs/concepts/overview/>.
- [4] *Cryptojacking: Case Study on Tesla's Experience*. URL: <https://www.centreforcybersecurity.com/en-sg/post/cryptojacking-case-study-on-teslas-experience>.
- [5] *Kubernetes Hardening Guide*. Cybersecurity Technical Report U/OO/168286-21. National Security Agency (NSA), Cybersecurity, and Infrastructure Security Agency (CISA), Aug. 2022, p. 59.
- [6] *OWASP Foundation, the Open Source Foundation for Application Security | OWASP Foundation*. URL: <https://owasp.org/>.
- [7] MinSu Chae, HwaMin Lee, and Kiyeol Lee. “A performance comparison of linux containers and virtual machines using Docker and KVM”. In: *Cluster Computing* 22.1 (Jan. 2019), pp. 1765–1775. ISSN: 1386-7857. DOI: [10.1007/s10586-017-1511-2](https://doi.org/10.1007/s10586-017-1511-2). URL: <https://doi.org/10.1007/s10586-017-1511-2>.
- [8] Robert P. Goldberg. “Survey of virtual machine research”. In: *Computer* 7.6 (1974), pp. 34–45. DOI: [10.1109/MC.1974.6323581](https://doi.org/10.1109/MC.1974.6323581).
- [9] *Oracle VirtualBox*. URL: <https://www.virtualbox.org/>.
- [10] *Hyper-V technology overview*. URL: <https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/hyper-v-overview>.
- [11] Robert P. Goldberg. “Survey of virtual machine research”. In: *Computer* 7.6 (1974), pp. 34–45. DOI: [10.1109/MC.1974.6323581](https://doi.org/10.1109/MC.1974.6323581).
- [12] *What is a Container?* Docker. URL: <https://www.docker.com/resources/what-container/>.
- [13] *What is Docker?* Docker Documentation. 200. URL: <https://docs.docker.com/get-started/docker-overview/>.
- [14] *What Would a Kubernetes 2.0 Look Like*. matduggan.com. URL: <https://matduggan.com/what-would-a-kubernetes-2-0-look-like/>.
- [15] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. “Large-scale cluster management at Google with Borg”. In: *Proceedings of the Tenth European Conference on Computer Systems*. Google. 2015, pp. 1–17.

- [16] Learn Kubernetes Security, Kaizhe Huang, and Pranjal Jumde. *Learn Kubernetes Security*. Packt Publishing, July 2020. 330 pp. URL: <https://learning.oreilly.com/library/view/learn-kubernetes-security/9781839216503/>.
- [17] *Scaling Kubernetes to 7,500 nodes*. URL: <https://openai.com/index/scaling-kubernetes-to-7500-nodes/>.
- [18] *Strimzi - Apache Kafka on Kubernetes*. URL: <https://strimzi.io/>.
- [19] *Home - Trivy Operator*. URL: <https://aquasecurity.github.io/trivy-operator/latest/>.
- [20] *What is DevOps?* Amazon Web Services, Inc. URL: <https://aws.amazon.com/devops/what-is-devops/>.
- [21] *DevOps Lifecycle : Different Phases in DevOps*. en-US. URL: <https://www.browserstack.com/guide/devops-lifecycle>.
- [22] Rice Liz and Hausenblas Michael. *Kubernetes Security*.
- [23] Joint Task Force. *Security and Privacy Controls for Information Systems and Organizations*. Tech. rep. 800-53, Revision 5. Includes updates as of December 10, 2020. Patch Release 5.1.1 issued November 7, 2023, with minor edits and new controls. National Institute of Standards and Technology, Sept. 2020. URL: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>.
- [24] Sara Motiee, Kirstie Hawkey, and Konstantin Beznosov. “Do windows users follow the principle of least privilege? investigating user account control practices”. In: *Proceedings of the Sixth Symposium on Usable Privacy and Security*. SOUPS ’10. Redmond, Washington, USA: Association for Computing Machinery, 2010. ISBN: 9781450302647. DOI: [10.1145/1837110.1837112](https://doi.org/10.1145/1837110.1837112). URL: <https://doi.org/10.1145/1837110.1837112>.
- [25] Creane Brendan and Gupta Amit. *Kubernetes Security and Observability*. O’Reilly Media, Oct. 2021. 192 pp.
- [26] Pratyusa K. Manadhata and Jeannette M. Wing. “An Attack Surface Metric”. In: *IEEE Transactions on Software Engineering* 37.3 (2011), pp. 371–386. DOI: [10.1109/TSE.2010.60](https://doi.org/10.1109/TSE.2010.60).
- [27] Abdallah Dawoud, Soeren Finster, Nicolas Coppik, and Virendra Ashiwal. “Better Left Shift Security! Framework for Secure Software Development”. In: 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). ISSN: 2768-0657. July 2024, pp. 642–649. DOI: [10.1109/EuroSPW61312.2024.00078](https://doi.org/10.1109/EuroSPW61312.2024.00078). URL: <https://ieeexplore.ieee.org/document/10628543/?arnumber=10628543>.

- [28] Murugiah Souppaya, John Morello, and Karen Scarfone. *Application container security guide*. National Institute of Standards and Technology, Sept. 25, 2017, NIST SP 800–190. DOI: [10.6028/NIST.SP.800-190](https://doi.org/10.6028/NIST.SP.800-190). URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>.
- [29] *What is MITRE ATT&CK: An Explainer*. Exabeam. URL: <https://www.exabeam.com/explainers/mitre-attck/what-is-mitre-attck-an-explainer/>.
- [30] Yossi Weizman. *Threat matrix for Kubernetes*. Microsoft Security Blog. Apr. 2, 2020. URL: <https://www.microsoft.com/en-us/security/blog/2020/04/02/attack-matrix-kubernetes/>.
- [31] Andong Chen, Zhaoxuan Jin, Ziyi Guo, and Yan Chen. *Breaking the Bulkhead: Demystifying Cross-Namespaces Reference Vulnerabilities in Kubernetes Operators*. 2025. arXiv: [2507.03387](https://arxiv.org/abs/2507.03387) [cs.CR]. URL: <https://arxiv.org/abs/2507.03387>.
- [32] Arsh Modak, S. D. Chaudhary, P. S. Paygude, and S. R. Ldate. “Techniques to Secure Data on Cloud: Docker Swarm or Kubernetes?” In: *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. 2018, pp. 7–12. DOI: [10.1109/ICICCT.2018.8473104](https://doi.org/10.1109/ICICCT.2018.8473104).
- [33] Md. Shazibul Islam Shamim, Farzana Ahamed Bhuiyan, and Akond Rahman. “XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices”. In: *2020 IEEE Secure Development (SecDev)*. 2020, pp. 58–64. DOI: [10.1109/SecDev45635.2020.00025](https://doi.org/10.1109/SecDev45635.2020.00025).
- [34] Kavindu Gunathilake and Indrajith Ekanayake. “K8s Pro Sentinel: Extend Secret Security in Kubernetes Cluster”. In: *2024 9th International Conference on Information Technology Research (ICITR)*. IEEE, Dec. 2024, pp. 1–5. DOI: [10.1109/icitr64794.2024.10857769](https://doi.org/10.1109/icitr64794.2024.10857769). URL: <http://dx.doi.org/10.1109/ICITR64794.2024.10857769>.
- [35] Garsha Rostami. “Role-Based Access Control (RBAC) Authorization in Kubernetes”. In: *Journal of ICT Standardization* 11.3 (2023), pp. 237–260. DOI: [10.13052/jicts2245-800X.1132](https://doi.org/10.13052/jicts2245-800X.1132).
- [36] Daniele Bringhenti, Riccardo Sisto, and Fulvio Valenza. “Security automation for multi-cluster orchestration in Kubernetes”. In: *2023 IEEE 9th International Conference on Network Softwarization (NetSoft)*. 2023, pp. 480–485. DOI: [10.1109/NetSoft57336.2023.10175419](https://doi.org/10.1109/NetSoft57336.2023.10175419).
- [37] Jacopo Bufalino, Jose Luis Martin-Navarro, Aleksi Peltonen, and Tuomas Aura. “Helm-ET: Reducing Exposure to Lateral Movement in Kubernetes Artifacts”. In: *IEEE 18th International Conference on Cloud Computing (CLOUD)*. 2025.
- [38] *neuvector/neuvector*. original-date: 2021-12-17T20:15:03Z. URL: <https://github.com/neuvector/neuvector>.

- [39] Yifan Li, Xiaohe Hu, Chengjun Jia, Kai Wang, and Jun Li. “Kano: Efficient Cloud Native Network Policy Verification”. In: *IEEE Transactions on Network and Service Management* 20.3 (2023), pp. 3747–3764. DOI: [10.1109/TNSM.2022.3229675](https://doi.org/10.1109/TNSM.2022.3229675).
- [40] Shunbin Dong, Yumin Xie, Jin Zhao, and Kun Qiu. “NPV: Fast Network Policy Verification for Cloud-Native Networking”. In: *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. 2024, pp. 461–472. DOI: [10.1109/ICDCS60910.2024.00050](https://doi.org/10.1109/ICDCS60910.2024.00050).
- [41] Hamed Hamzeh, Sofia Meacham, and Kashaf Khan. “A New Approach to Calculate Resource Limits with Fairness in Kubernetes”. In: *2019 First International Conference on Digital Data Processing (DDP)*. 2019, pp. 51–58. DOI: [10.1109/DDP.2019.00020](https://doi.org/10.1109/DDP.2019.00020).
- [42] *Kubernetes Admission Controllers: A Fast-Track Guide* Wiz. wiz.io. URL: <https://www.wiz.io/academy/kubernetes-admission-controllers>.
- [43] *Admission Controllers 101*. Kyverno. URL: <https://kyverno.io/docs/introduction/admission-controllers/>.
- [44] Noah Spahn, Nils Hanke, Thorsten Holz, Christopher Kruegel, and Giovanni Vigna. “Container Orchestration HoneyPot: Observing Attacks in the Wild”. In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. RAID ’23*. Hong Kong, China: Association for Computing Machinery, 2023, pp. 381–396. ISBN: 9798400707650. DOI: [10.1145/3607199.3607205](https://doi.org/10.1145/3607199.3607205). URL: <https://doi.org/10.1145/3607199.3607205>.
- [45] Chakshu Gupta, Thijs Van Ede, and Andrea Continella. “HoneyKube: Designing and Deploying a Microservices-based Web HoneyPot”. In: *2023 IEEE Security and Privacy Workshops (SPW)*. 2023, pp. 1–11. DOI: [10.1109/SPW59333.2023.00005](https://doi.org/10.1109/SPW59333.2023.00005).
- [46] Rahman Akond, Shamim Shazibul Islam, Bose Dibyendu Brinto, and Pandita Rahul. “Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study”. In: (May 26, 2023), p. 36.
- [47] Jacopo Bufalino, Jose Luis Martin-Navarro, Mario Di Francesco, and Tuomas Aura. *Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations*. 2025. arXiv: [2506.21134](https://arxiv.org/abs/2506.21134) [cs.CR]. URL: <https://arxiv.org/abs/2506.21134>.
- [48] Eunsook Kim, Kyungwoon Lee, and Chuck Yoo. “On the Resource Management of Kubernetes”. In: *2021 International Conference on Information Networking (ICOIN)*. 2021, pp. 154–158. DOI: [10.1109/ICOIN50884.2021.9333977](https://doi.org/10.1109/ICOIN50884.2021.9333977).

- [49] Tan N. Le, Xiao Sun, Mosharaf Chowdhury, and Zhenhua Liu. “AlloX: compute allocation in hybrid clusters”. In: *Proceedings of the Fifteenth European Conference on Computer Systems*. EuroSys ’20. Heraklion, Greece: Association for Computing Machinery, 2020. ISBN: 9781450368827. DOI: [10.1145/3342195.3387547](https://doi.org/10.1145/3342195.3387547). URL: <https://doi.org/10.1145/3342195.3387547>.
- [50] Zhang Wei-guo, Ma Xi-lin, and Zhang Jin-zhong. “Research on Kubernetes’ Resource Scheduling Scheme”. In: *Proceedings of the 8th International Conference on Communication and Network Security*. ICCNS ’18. Qingdao, China: Association for Computing Machinery, 2018, pp. 144–148. ISBN: 9781450365673. DOI: [10.1145/3290480.3290507](https://doi.org/10.1145/3290480.3290507). URL: <https://doi.org/10.1145/3290480.3290507>.
- [51] Dongmin Choi, Hyunmin Seo, Kwanwoo Kim, Myoungsung You, Seungwon Shin, and Jinwoo Kim. “Uncovering Threats in Container Systems: A Study on Misconfigured Container Components in the Wild”. In: *IEEE Access* 12 (2024), pp. 192931–192945. DOI: [10.1109/ACCESS.2024.3514751](https://doi.org/10.1109/ACCESS.2024.3514751).
- [52] Hui Zhu and Christian Gehrman. “Kub-Sec, an automatic Kubernetes cluster AppArmor profile generation engine”. In: *2022 14th International Conference on COMMunication Systems NETWORKS (COMSNETS)*. 2022, pp. 129–137. DOI: [10.1109/COMSNETS53615.2022.9668504](https://doi.org/10.1109/COMSNETS53615.2022.9668504).
- [53] *Dynamic Threat Analysis for Containers (DTA) - Aqua Security*. Aqua. Apr. 12, 2020. URL: <https://www.aquasec.com/products/container-analysis/>.
- [54] *Container Threat Detection overview*. Security Command Center. Google Cloud. URL: <https://cloud.google.com/security-command-center/docs/concepts-container-threat-detection-overview>.
- [55] *Container Threat Detection*. Sysdig. URL: <https://sysdig.com/learn-cloud-native/what-is-container-threat-detection/>.
- [56] Pedro Horschulhack, Eduardo K. Viegas, Altair O. Santin, Felipe V. Ramos, and Pietro Tedeschi. “Detection of quality of service degradation on multi-tenant containerized services”. In: *Journal of Network and Computer Applications* 224 (2024), p. 103839. ISSN: 1084-8045. DOI: <https://doi.org/10.1016/j.jnca.2024.103839>. URL: <https://www.sciencedirect.com/science/article/pii/S108480452400016X>.
- [57] Josue Genaro Almaraz-Rivera. “An Anomaly-based Detection System for Monitoring Kubernetes Infrastructures”. In: *IEEE Latin America Transactions* 21.3 (2023), pp. 457–465. DOI: [10.1109/TLA.2023.10068850](https://doi.org/10.1109/TLA.2023.10068850).
- [58] Jim Angel, Pushkar Joglekar, and Savitha Raghunathan. *A Closer Look at NSA/CISA Kubernetes Hardening Guidance*. Kubernetes. Oct. 5, 2021. URL: <https://kubernetes.io/blog/2021/10/05/nsa-cisa-kubernetes-hardening-guidance/>.

- [59] *Auditing*. Kubernetes. URL: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>.
- [60] Center for Internet Security. *CIS Kubernetes Benchmark*. Sept. 23, 2024, p. 328.
- [61] Amit Hendre and Karuna Pande Joshi. “A Semantic Approach to Cloud Security and Compliance”. In: *2015 IEEE 8th International Conference on Cloud Computing*. 2015, pp. 1081–1084. DOI: [10.1109/CLOUD.2015.157](https://doi.org/10.1109/CLOUD.2015.157).
- [62] *Security Command Center*. Google Cloud. URL: <https://cloud.google.com/security/products/security-command-center>.
- [63] *AWS Config Documentation*. URL: <https://docs.aws.amazon.com/config/>.
- [64] *Azure Policy documentation*. URL: <https://learn.microsoft.com/en-us/azure/governance/policy/>.
- [65] *Prisma Cloud Enterprise Edition*. URL: <https://docs.prismacloud.io/en/enterprise-edition>.
- [66] *Security and compliance | OpenShift Container Platform | 4.13 | Red Hat Documentation*. URL: https://docs.redhat.com/en/documentation/openshift_container_platform/4.13/html/security_and_compliance/index.
- [67] Purnima Padmanabhan. *Introducing VMware Aria*. VMware Cloud Management. Aug. 30, 2022. URL: <https://blogs.vmware.com/management/2022/08/introducing-vmware-aria.html>.
- [68] Ghareeb Falazi, Lukas Harzenetter, Kálmán Képes, Frank Leymann, Uwe Breitenbücher, Evangelos Ntontos, Uwe Zdun, Martin Becker, and Elena Heldwein. “Compliance Management of IaC-Based Cloud Deployments During Runtime”. In: *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing*. UCC ’23. Taormina (Messina), Italy: Association for Computing Machinery, 2024. ISBN: 9798400702341. DOI: [10.1145/3603166.3632135](https://doi.org/10.1145/3603166.3632135). URL: <https://doi-org.libproxy.aalto.fi/10.1145/3603166.3632135>.
- [69] Mozhdeh Farhadi, Guillaume Pierre, and Daniele Miorandi. “Towards automated privacy compliance checking of applications in Cloud and Fog environments”. In: *2021 8th International Conference on Future Internet of Things and Cloud (FiCloud)*. 2021, pp. 11–18. DOI: [10.1109/FiCloud49777.2021.00010](https://doi.org/10.1109/FiCloud49777.2021.00010).
- [70] Mehdi Bahrami, Abhishek Malvankar, Karan K. Budhraj, Chinmay Kundu, Mukesh Singhal, and Ashish Kundu. “Compliance-Aware Provisioning of Containers on Cloud”. In: *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. 2017, pp. 696–700. DOI: [10.1109/CLOUD.2017.95](https://doi.org/10.1109/CLOUD.2017.95).

- [71] M.R. Stytz. “Considering defense in depth for software applications”. In: *IEEE Security Privacy* 2.1 (2004), pp. 72–75. DOI: [10.1109/MSECP.2004.1264860](https://doi.org/10.1109/MSECP.2004.1264860).
- [72] Zlatan Morić, Vedran Dakić, and Tomislav Čavala. “Security Hardening and Compliance Assessment of Kubernetes Control Plane and Workloads”. In: *Journal of Cybersecurity and Privacy* 5.2 (2025). ISSN: 2624-800X. DOI: [10.3390/jcp5020030](https://doi.org/10.3390/jcp5020030). URL: <https://www.mdpi.com/2624-800X/5/2/30>.
- [73] *GoogleContainerTools/distroless*. original-date: 2017-04-18T22:02:38Z. URL: <https://github.com/GoogleContainerTools/distroless>.
- [74] *Harbor*. URL: <https://harbor.cs.aalto.fi>.
- [75] *Trivy*. en. URL: <https://trivy.dev/v0.63/>.
- [76] *Kube-bench*. URL: <https://aquasecurity.github.io/kube-bench/v0.6.15/>.
- [77] Xiaoying Bai, W.T. Tsai, R. Paul, Techeng Shen, and Bing Li. “Distributed end-to-end testing management”. In: *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*. 2001, pp. 140–151. DOI: [10.1109/EDOC.2001.950430](https://doi.org/10.1109/EDOC.2001.950430).

A Resource Management for Pods

```
spec:
  containers:
  - image: nginx
    imagePullPolicy: Always
    name: resource-control
    resources:
      requests:
      limits:
        hugepages-2Mi: 100Mi
      requests:
        cpu: 500m
        memory: 300Mi
        hugepages-2Mi: 100Mi
```

Listing 12: Pod resource limit [16]

B LimitRanger

```
apiVersion: "v1"
kind: "LimitRange"
metadata:
  name: limit-demo
  namespace: demo
spec:
  limits:
  - type: "Container"
    max:
      memory: 512Mi
      cpu: 500m
    min:
      memory: 50Mi
      cpu: 50m
```

Listing 13: LimitRange configuration on namespace [16]

C Network Policies

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: access-policy
spec:
  podSelector:
    matchLabels:
      app: access
  ingress:
  - from:
    - ipBlock:
```

```

        cidr: 0.0.0.0/0
    ports:
    - port: 80
      protocol: TCP
egress:
- to:
  - ipBlock:
      cidr: 0.0.0.0/0
- to:
  - podSelector:
      matchLabels:
        app: wiki
- to:
  - podSelector:
      matchLabels:
        strimzi.io/kind: Kafka
    ports:
    - port: 9093
- to:
  - namespaceSelector: {}
    podSelector:
      matchLabels:
        k8s-app: kube-dns
    ports:
    - port: 53
      protocol: UDP

```

Listing 14: Access network policy

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: dealer-policy
spec:
  podSelector:
    matchLabels:
      app: dealer
  ingress: []
  # egress:
  # - {}
  egress:
  - to:
    - podSelector:
        matchLabels:
          strimzi.io/kind: Kafka
      ports:
      - port: 9093
  - to:
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
      ports:
      - port: 53
        protocol: UDP

```

Listing 15: Dealer network policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: guard-policy
spec:
  podSelector:
    matchLabels:
      app: guard
  ingress: []
  egress:
    - to:
        - podSelector:
            matchLabels:
              strimzi.io/kind: Kafka
      ports:
        - port: 9093
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - port: 53
        protocol: UDP
```

Listing 16: Guard network policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mongo-policy
spec:
  podSelector:
    matchLabels:
      app: mongo
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: wiki
  egress:
```

Listing 17: Mongo network policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: user-policy
spec:
  podSelector:
    matchLabels:
      app: user
```



```

policyTypes:
  - Ingress
  - Egress
ingress:
  - from:
      - ipBlock:
          cidr: 0.0.0.0/0
        ports:
          - port: 80
            protocol: TCP
# egress:
#   - {}
egress:
  - to:
      - ipBlock:
          cidr: 0.0.0.0/0
    - to:
        - podSelector:
            matchLabels:
              strimzi.io/kind: Kafka
          ports:
            - port: 9093
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
          ports:
            - port: 53
              protocol: UDP

```

Listing 18: User network policy

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: wiki-policy
spec:
  podSelector:
    matchLabels:
      app: wiki
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - ipBlock:
            cidr: 0.0.0.0/0
          ports:
            - port: 80
              protocol: TCP
    - from:
        - podSelector:
            matchLabels:
              app: access
          ports:

```

```
        - port: 80
# egress:
# - {}

egress:
- to:
  - namespaceSelector: {}
    podSelector:
      matchLabels:
        k8s-app: kube-dns
    ports:
      - port: 53
        protocol: UDP
- to:
  - podSelector:
      matchLabels:
        app: mongo
```

Listing 19: Wiki Pod network policy